Chapter 1.Convolution operators and kernel operators

ARIN7013 Numerical Methods in Artificial Intelligence

The University of Hong Kong

Table of Contents

1.1 Convolutional Neural network

Introduction

- LeCun introduced the convolutional networks in 1989, which is also known as convolutional neural networks (CNN). They are a specialized kind of neural network for processing data that has a known grid-like topology.
- Examples inlude time-series data, which can be thought of as a 1-D grid taking samples at regular time intervals, and image data, which can be thought of as a 2-D grid of pixels.
- CNNs use convolution in place of general matrix multiplication in at least one layer.
- Convolutional networks have been successful in practical applications.

This Section is based on Chapter 9 of the book *Deep Learning* by Goodfellow, Bengio and Courville (2016).

Variant of 2-d convolution: Cross-correlation

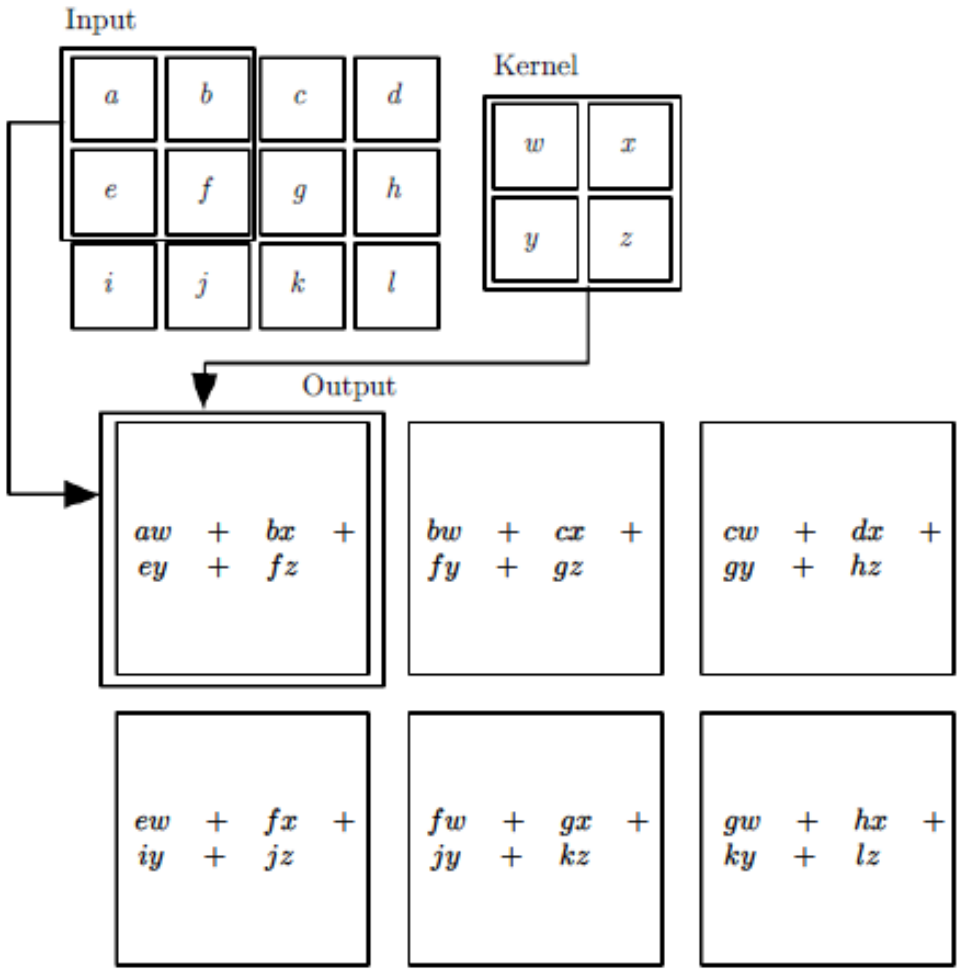
The commutative property of convolution arises because we have fipped the kernel w.r.t. the input. This property is useful for writing proofs, but not an important property of a neural network implementation. Instead, many neural network libraries implement a related function called the *cross-correlation*, which is the same as convolution but without flipping the kernel,

Cross-correlation

Let $I \in \mathbb{R}^{m_1 \times n_1}$, $K \in \mathbb{R}^{m_2 \times n_2}$, and let $S \in \mathbb{R}^{(m_1 - m_2 + 1) \times (n_1 - n_2 + 1)}$ be

$$S(i,j) := (I * K)(i,j) = \sum_{k_1=1}^{m_2} \sum_{k_2=1}^{n_2} I(i + k_1, j + k_2) K(k_1, k_2).$$

Example: 2-d cross-correlation



Convolution[1] in machine learning applications

input is usually a multidimensional array (or *tensor*) of data

kernel is usually a tensor of parameters adapted by the learning algorithm

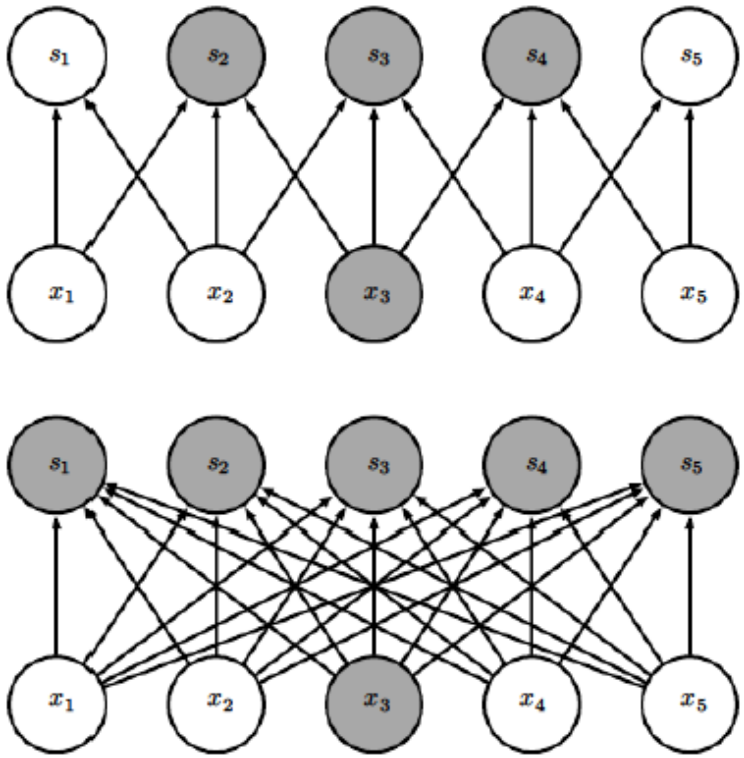
advantages of convolution in ML

- sparse interaction\sparse connectivity\sparse weights
- parameter sharing
- equivariant representations

[1] We refer cross correlation as convolution in the CNNs

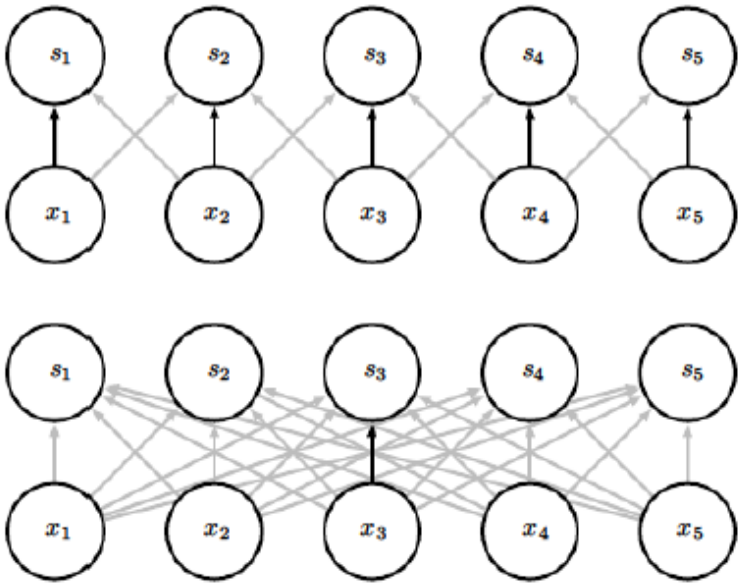
Example: sparse interaction

Figure: (Top) the output unit s is formed by convolution with a kernel of width 3, only 3 outputs are affected by the input unit x. (Bottom) s is formed by matrix multiplication, connectivity is no longer sparse, so all outputs are affected by x_3.



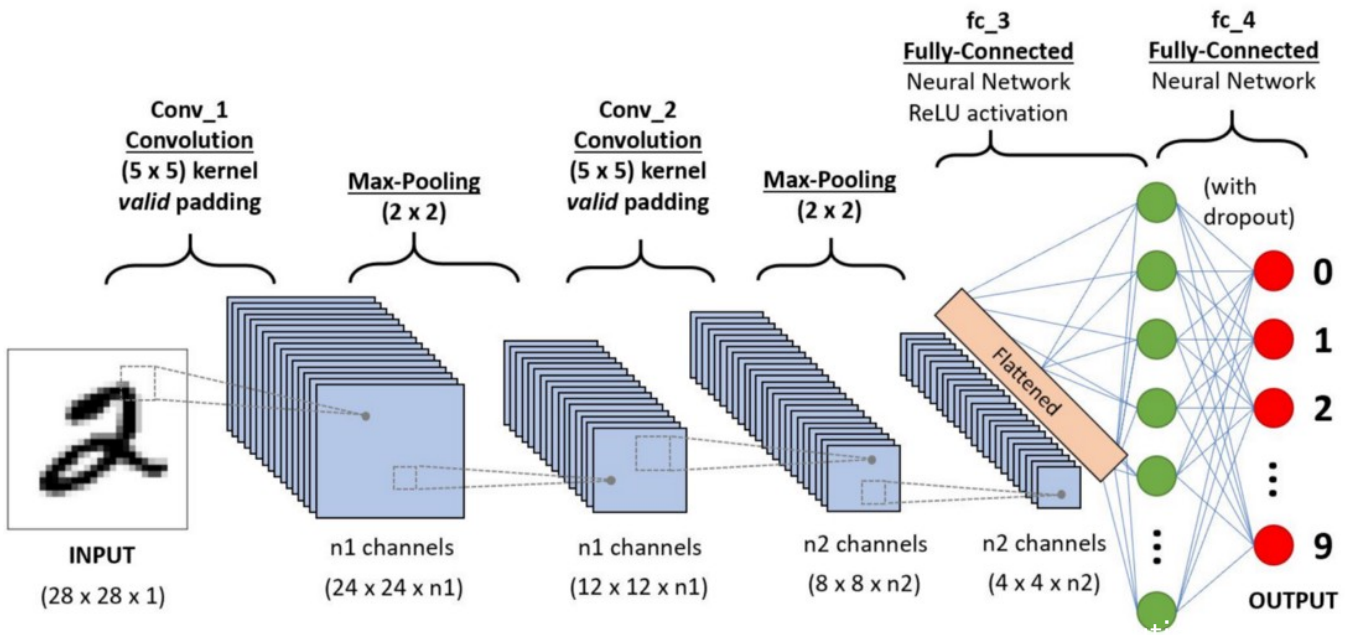
Example: parameter sharing

Figure: Block arrows indicate the connections that use a particular parameter in two different models. (Top) The black arrows indicate uses of the central element of a 3-element kernel in a convolutional model. Due to parameter sharing, this single parameter is used at all input locations. (Bottom) The singule black arrow indicates the use of the central element of the weight matrix in a fully connected model. This model has no parameter sharing so the parameter is used only once.



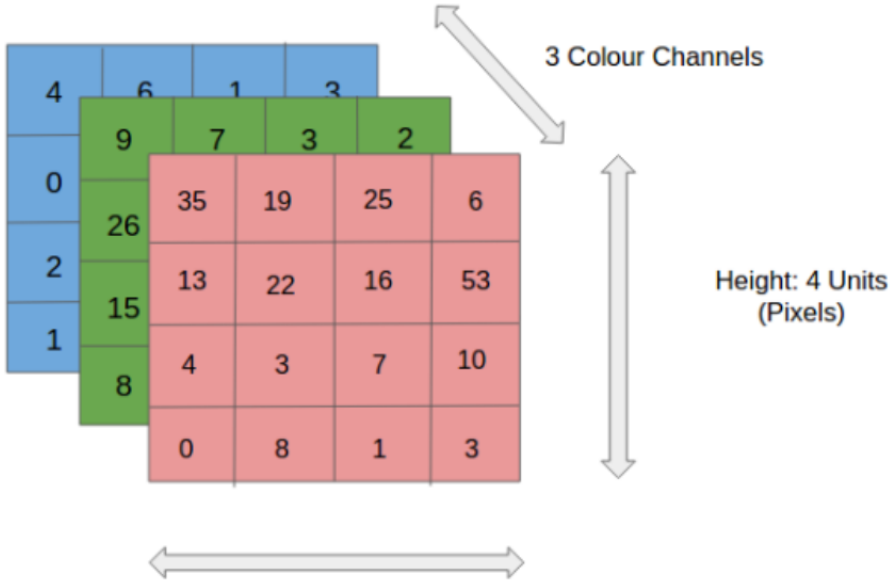
Architecture of the Convolutional Neural Network

- A Convolutional Neural Network (ConvNet/CNN) is a Deep Learning algorithm which can take in an input image, assign importance (learnable weights and biases) to various aspects/objects in the image and be able to differentiate one from the other.
- The architecture of a ConvNet is analogous to that of the connectivity pattern of Neurons in the Human Brain. Individual neurons respond to stimuli only in a restricted region of the visual field known as the Receptive Field.



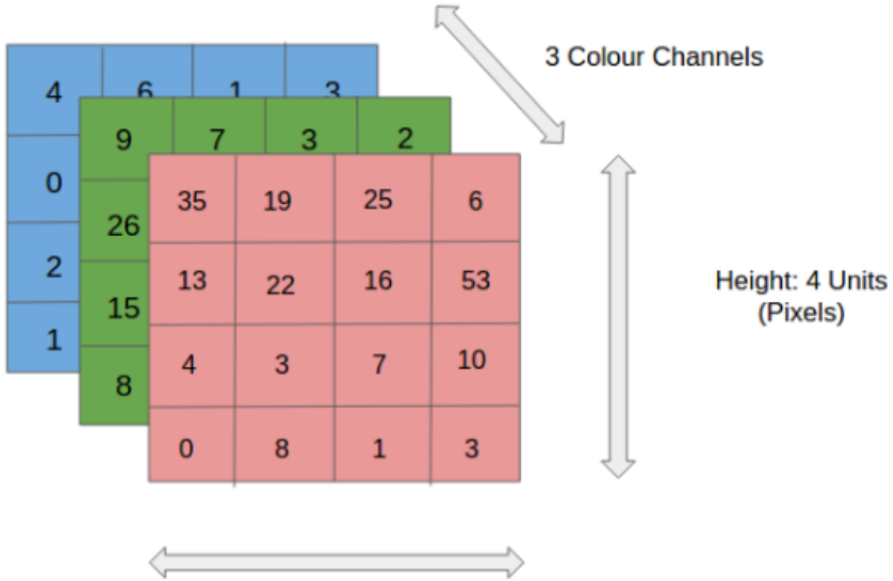
ConvNet for the RGB Image

- An image is nothing but a matrix of pixel values.
- A ConvNet is able to successfully capture the Spatial and Temporal dependencies in an image through the application of relevant filters.
- The architecture performs a better fitting to image dataset due to the reduction in the number of parameters involved and reusability of weights.
- The network can be trained to understand the sophistication of the image better.

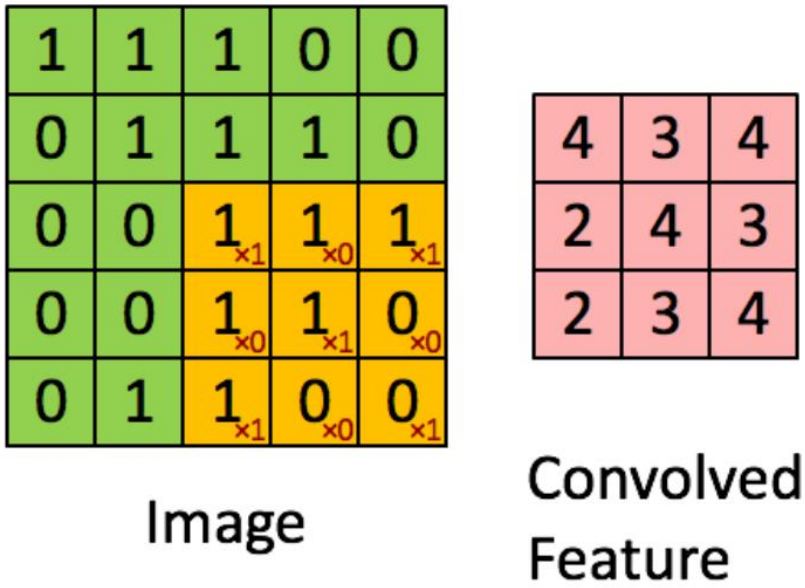


ConvNet for the RGB Image (cont.)

- In the figure, we have an RGB image which has been separated by its three color planes — Red, Green, and Blue.
- How computationally intensive things would get once the images reach dimensions, say 8K (7680×4320).
- ConvNet reduces the images into a form that is easier to process, without losing features that are critical for getting a good prediction.
- This is important when we are to design an architecture that is not only good at learning features but also is scalable to massive datasets.



Convolution Layer — The Kernel



Convoluting a 5×5×1 image with a 3×3×1 kernel to get a 3×3×1 convolved feature

- Image Dimensions = 5 (Height) x 5 (Breadth) x 1 (Number of channels, eg. RGB).
- In the above demonstration, the green section resembles our 5x5x1 input image, I.
- The element involved in carrying out the convolution operation in the first part of a Convolutional Layer is called the Kernel/Filter, K, represented in the color yellow. We have selected K as a 3x3x1 matrix.

Convolution Layer — The Kernel (cont.)

- The Kernel shifts 9 times because of $StrideLength = 1$ (Non-Strided), every time performing a matrix multiplication operation between K and the portion P of the image over which the kernel is hovering.
- The filter moves to the right with a certain Stride Value till it parses the complete width. Moving on, it hops down to the beginning (left) of the image with the same Stride Value and repeats the process until the entire image is traversed.

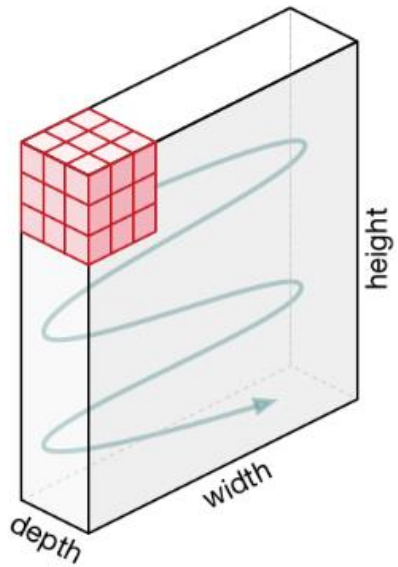


Figure: Movement of the Kernel

Multiple Input channels

- Color image may have three RGB channels
- Converting to one grayscale channel loses information

The Kernel for multiple channels

- In the case of images with multiple channels (e.g. RGB), the Kernel has the same depth as that of the input image.
- Matrix Multiplication is performed between Kn and In stack ([K1, I1]; [K2, I2]; [K3, I3]) and all the results are summed with the bias to give us a squashed one-depth channel Convoluted Feature Output.

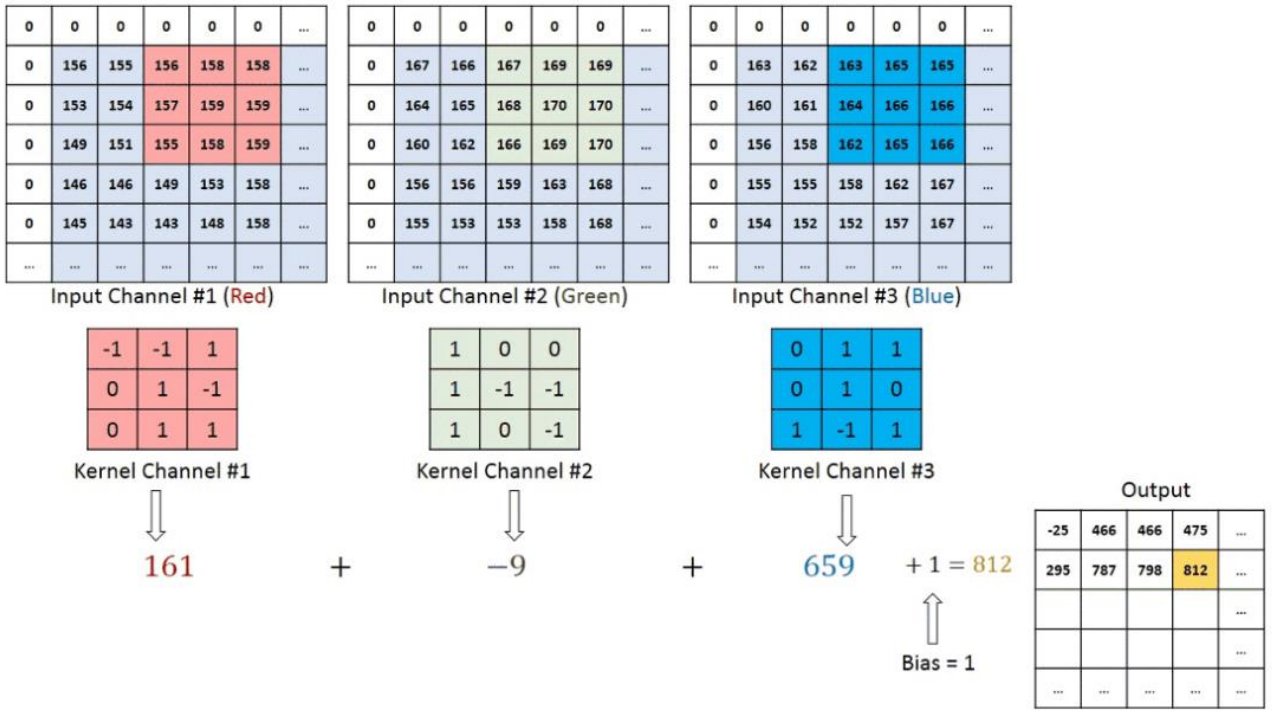


Figure: Convolution operation on a *MxNx3* image matrix with a 3x3x3 Kernel.

Extract features

- The objective of the Convolution Operation is to extract the high-level features such as edges, from the input image.
- ConvNets need not be limited to only one Convolutional Layer. Conventionally, the first ConvLayer is responsible for capturing the Low-Level features such as color, gradient orientation, etc.
- With added layers, the architecture adapts to the High-Level features as well, giving us a network which has the wholesome understanding of images in the dataset, similar to how we would.
- There are two types of results to the operation:
  1. The convolved feature is reduced in dimensionality as compared to the input (by applying Valid Padding);
  2. The dimensionality is either increased or remains the same (by applying Same Padding.

Same Padding and Valid Padding

- When we augment the 5x5x1 image into a 7x7x1 image and then apply the 3x3x1 kernel over it, we find that the convolved matrix turns out to be of dimensions 5x5x1. Hence the name — Same Padding.
- If we perform the same operation without padding, we are presented with a matrix which has dimensions of the Kernel (3x3x1) itself — Valid Padding.

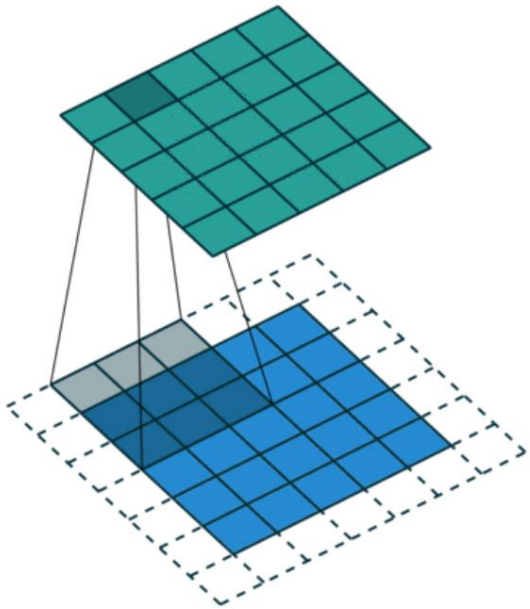


Figure: SAME padding: 5x5x1 image is padded with 0s to create a 6x6x1 image.

Pooling Layer: motivation

- A limitation of the feature map output of convolutional layers is that they record the precise position of features in the input. Therefore, small movement in the position of the feature in the input image results in a different feature map. This can happen with re-cropping, rotation, shifting and other minor changes to the input image.
- A common approach to addressing this problem from signal processing is "down sampling", i.e., a lower resolution version of an input signal is created that still contains the large or important structural elements, without the fine detail that may not be as useful to the task.
- Down sampling can be achieved with convolutional layers by changing the stride of the convolution across the image. A more robust and common approach is use a pooling layer.

What are Pooling layers?

- Similar to the Convolutional Layer, the Pooling layer is responsible for reducing the spatial size of the Convolved Feature, and decreasing the computational power required to process the data through dimensionality reduction.
- It is useful for extracting dominant features which are rotational and positional invariant, thus it can effectively train the model.
- The result of using a pooling layer and creating down sampled or pooled feature maps is a summarized version of the features detected in the input. They are useful as small changes in the location of the feature in the input detected by the convolutional layer will result in a pooled feature map with the feature in the same location. This capability added by pooling is called the model's invariance to local translation.

Max Pooling and Average Pooling

- There are two types of Pooling: Max Pooling and Average Pooling. Max Pooling returns the maximum value from the portion of the image covered by the Kernel. Average Pooling returns the average of all the values from the portion of the image covered by the Kernel.

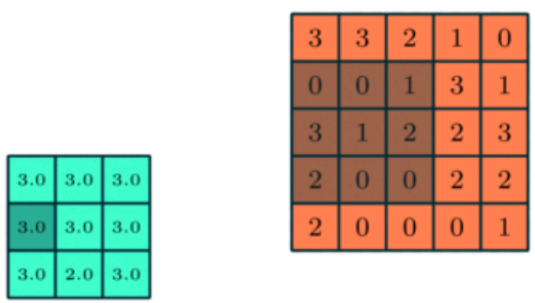
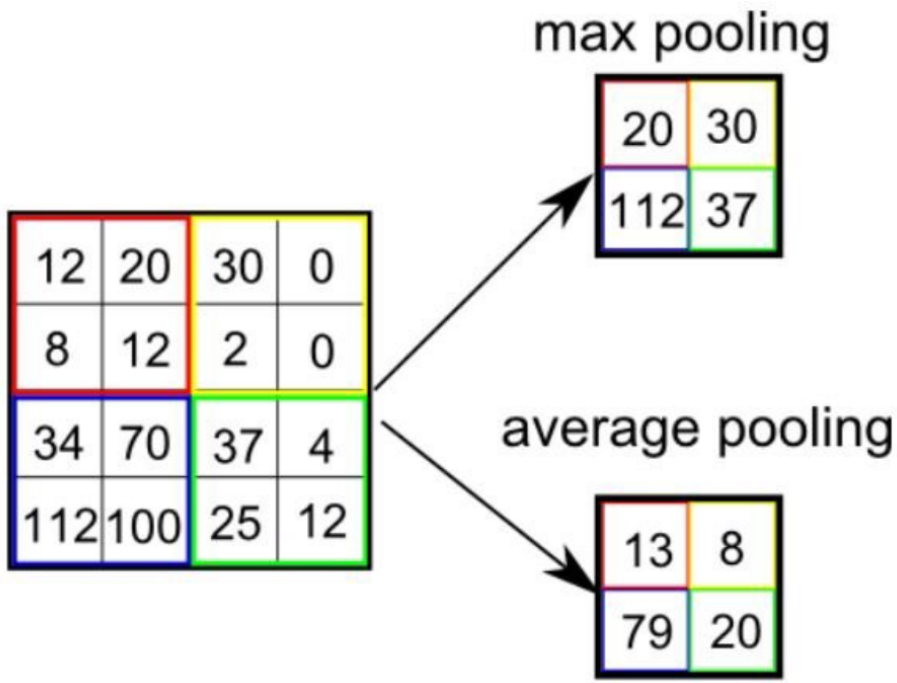


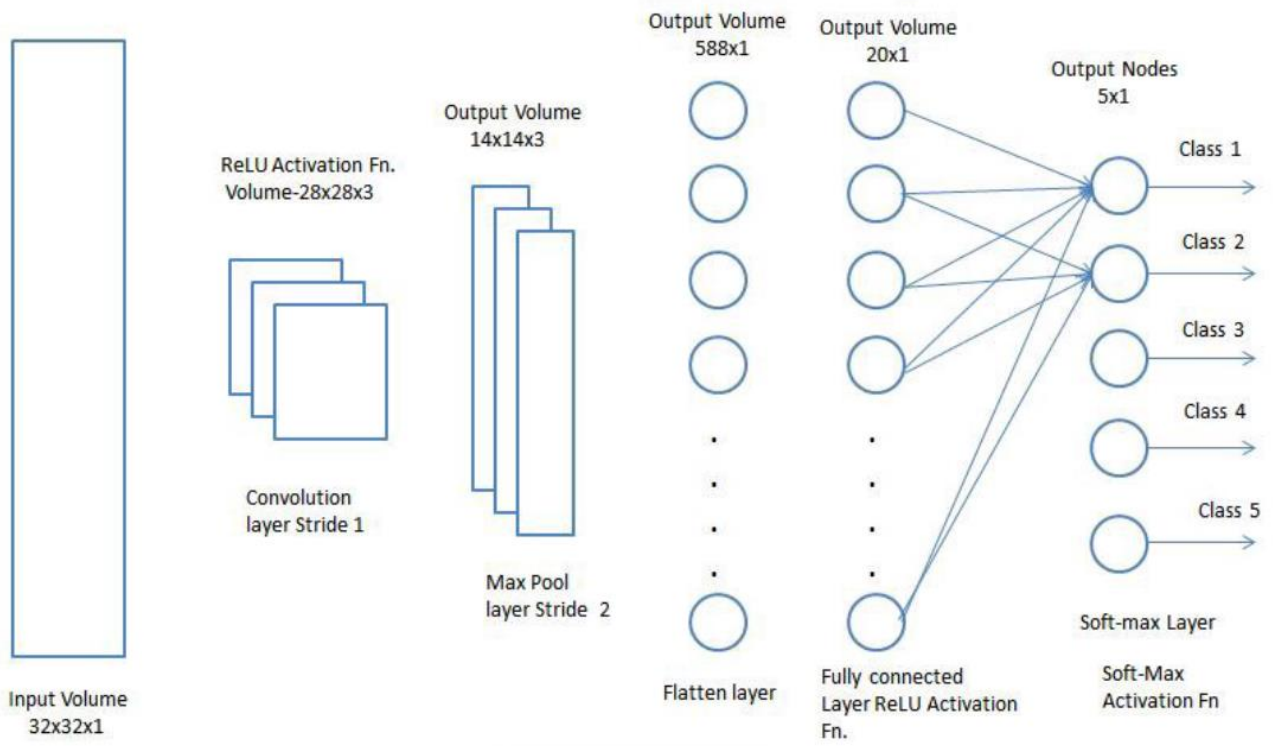
Figure: 3*x*3 pooling over 5*x*5 convolved feature.

Pooling Layer (cont.)

- Max Pooling also performs as a Noise Suppressant. It discards the noisy activations altogether and also performs de-noising along with dimensionality reduction.
- Average Pooling simply performs dimensionality reduction as a noise suppressing mechanism. Hence, we can say that Max Pooling performs a lot better than Average Pooling.
-



- The Convolutional Layer and the Pooling Layer, together form the i-th layer of a Convolutional Neural Network.
- Depending on the complexities in the images, the number of such layers may be increased for capturing low-levels details even further, but at the cost of more computational power.
- After going through the above process, one can enable the model to understand the features. Moving on, we are going to flatten the final output and feed it to a regular Neural Network for classification.



Pooling VS Convolution

- Pooling layers can apply similar padding and stride as convolutional layers
- Pooling has no kernel (or weights) to train

Some remarks

- Adding a Fully-Connected layer is a (usually) cheap way of learning non-linear combinations of the high-level features as represented by the output of the convolutional layer. The Fully-Connected layer is learning a possibly non-linear function in that space.
- Now that we have converted our input image into a suitable form for our Multi-Level Perceptron, we shall flatten the image into a column vector. The flattened output is fed to a feed-forward neural network and backpropagation applied to every iteration of training. Over a series of epochs, the model is able to distinguish between dominating and certain low-level features in images and classify them using the Softmax Classification technique.
- There are various architectures of CNNs available which have been key in building algorithms which power and shall power AI as a whole in the foreseeable future. Some of them have been listed below: LeNet, AlexNet, VGGNet, GoogLeNet, ResNet, ZFNet.

Summary: Convolutional Neural Network

- Many physical and scientific problems are defined on regular domains and can be discretized by using grids.
- Convolutional Neural Network (ConvNet/CNN) is an efficient Deep Learning algorithm to study computer vision problems. There are many architectures of CNN available that can reduce computational complexity while maintaining good performance.

1.2 Kernel methods

Overview

Motivation

- Develop versatile algorithms to process and analyze data... ...
- without making any assumptions regarding the type of data (vectors, strings, graphs, images, ...)

Approach

- Develop methods based on pairwise comparisons.
- By imposing constraints on the pairwise comparison function (positive definite kernels), we obtain a general framework for learning from data (optimization in Hilbert space).

Kernel method: Motivations

Two theoretical results underpin a family of powerful algorithms for data analysis using p.d. kernels, collectively known as kernel method,

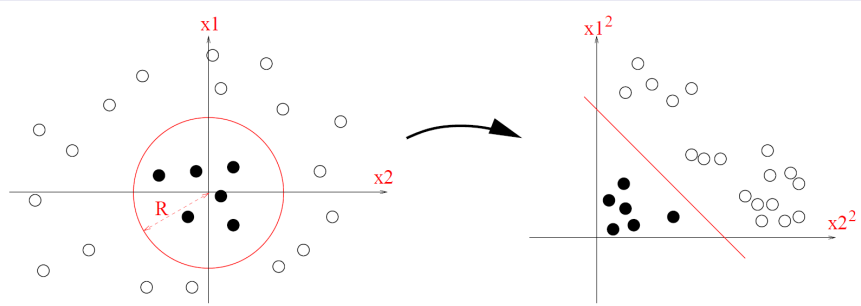
- The kernel trick, based on the representation of p.d. kernels as inner product
- The representer theorem, based on some properties of the regularization functionals defined by the Hilbert space norm.

The kernel map can embed the data into a high dimensional feature space, which has an inner product defined using the associated positive definite kernel. Then the kernel trick enables computationally efficient implementation of learning, without explicitly handling the high dimensional representation of the the domain instances.
Kernel methods enable to enrich the expressive power of simple predictor, e.g., the linear predictor.

Kernel method: Motivations

Example

How to separate the following training set using half spaces?



Halfspace = linear decision boundary in some dimension.
The problem = data not linearly separable in original space.
The fix = map to higher-dimensional space → becomes linearly separable → learn a halfspace there.

To make the class of halfspaces more expressive, we can first map the original instance space $\mathcal{X} := \mathbb{R}^2$ into a higher dimension space $\mathbb{R}^4$ and then learn a halfspace in that space.

basic paradigm of kernel method

Given some domain set $\mathcal{X}$, we aim to learn $h : \mathcal{X} \to \mathbb{R}$ using certain training data.

1: choose a mapping $\psi : \mathcal{X} \to \mathcal{H}$ for some feature space $\mathcal{H}$, that will usually be $\mathbb{R}^n$ for some n (however, the range of such a mapping can be any Hilbert space, including such spaces of infinite dimension)
2: Given a set of labelled training data $S := \{(x_i, y_i)\}$ for $i = 1, \cdots, m$, create the image sequences $\hat{S} = \{(\psi(x_i), y_i)\}$ for $i = 1, \cdots, m$.
3: Train a linear predictor $\hat{h} : \mathcal{H} \to \mathbb{R}$ over $\hat{S}$.
4: Predict the label of a test point x to be $\hat{h}(\psi(x))$.

The success of this learning paradigm depends on choosing a good map ψ for a given learning task.

Positive Definite (p.d.) Kernels

Definition

A positive definite (p.d.) kernel on a set χ is a function $K : \chi \times \chi \to \mathbb{R}$ that is symmetric:

$$\forall\, (x, x') \in \chi^2 : K(x, x') = K(x', x),$$

and which satisfies, all $N \in \mathbb{N}$, $(x^1, \cdots, x^N) \in \chi^N$ and $(a_1, \cdots, a_N) \in \mathbb{R}^N$,

$$\sum_{i=1}^{N} \sum_{j=1}^{N} a_i a_j K(x^i, x^j) \geq 0.$$

这里的 **"all $N \in \mathbb{N}$"** 意思是：

> **不管你选多少个点 ($N = 1, 2, 3, ...$)，下面这个非负性条件都必须成立。**

更具体地说：

1. 你先随便取一个正整数 N(表示你要抽取 N 个样本点)。
2. 再从集合 $\mathcal{X}$ 里随便挑 N 个点：$x^1, ..., x^N$。
3. 再随便选一组实数系数：$a_1, ..., a_N$。
4. 形成这个二次型：

$$\sum_{i=1}^{N} \sum_{j=1}^{N} a_i a_j K(x^i, x^j) \geq 0$$

这个必须永远 ≥ 0。

Similarity matrices of p.d. kernels

Remarks

- Equivalently, a kernel K is p.d. if and only if, for any $N \in \mathbb{N}$ and any set of points $(x^1, x^2, \cdots, x^N) \in \chi^N$, the similarity matrix $[K]_{ij} := K(x^i, x^j)$ is positive semidefinite.
- Kernel methods are algorithms that take such matrices as input.

把 Gram 矩阵写出来：

$K_N = [K(x^i, x^j)]_{i,j=1}^N$

那上面那坨就是

$a^\top K_N a \geq 0$

所以 "对所有 N" 等价于：

> 对任意大小的 Gram 矩阵 K_N，它都必须是半正定 (PSD)。

The simplest p.d. kernel, for real numbers

Example

Let $\chi = \mathbb{R}$. The function $K : \chi^2 \to \mathbb{R}$ defined by:

$$\forall (x, x') \in \chi^2 : \; K(x, x') = xx'$$

is p.d.

The simplest p.d. kernel, for vectors

Example

Let $\chi = \mathbb{R}^d$. The function $K : \chi^2 \to \mathbb{R}$ defined by:

$$\forall (x, x') \in \chi^2 : \; K(x, x') = \langle x, x' \rangle_\chi$$

is p.d., (it is often called the linear kernel).

A more ambitious p.d. kernel

Example

Let χ be any set and $\phi : \chi \to \mathbb{R}^d$. The function $K : \chi^2 \to \mathbb{R}$ defined by:

$$\forall (x, x') \in \chi^2 : \; K(x, x') = \langle \phi(x), \phi(x') \rangle_{\mathbb{R}^d}$$

is p.d.

Polynomial kernel

Example

Let $\chi := \mathbb{R}^2$ and $\phi : \chi \to \mathbb{R}^3$ defined by

$$\phi(x_1, x_2) = (x_1^2, \sqrt{2}x_1x_2, x_2^2),$$

then the function $K : \chi^2 \to \mathbb{R}$ defined by:

$$\forall (x, x') \in \chi^2 : \; K(x, x') = \langle \phi(x), \phi(x') \rangle_{\mathbb{R}^3} = \langle x, x' \rangle_{\mathbb{R}^2}^2$$

is a p.d. kernel.

Combining kernels

Theorem

- *If K_1 and K_2 are p.d. kernels, then*
  - $K_1 + K_2$
  - $K_1 K_2$
  - cK_1 *for* $c \geq 0$

  are also p.d. kernels.
- *If $\{K_i\}_{i \geq 1}$ is a sequence of p.d. kernels that converges pointwisely to a function K, i.e.,*

$$\forall (x, x') \in \mathcal{X}^2, K(x, x') = \lim_{n \to \infty} K_i(x, x'),$$

then K is also a p.d. kernel.

Proof

K_1K_2 is a p.d. kernel.

Consider n points in $\mathcal{X}$ and the corresponding $n \times n$ psd similarity matrices $\widehat{K}_1$ and $\widehat{K}_2$ for K_1 and K_2. Assume that they admit factorization of $\widehat{K}_1 = X^T X$ and $\widehat{K}_2 = Y^T Y$. Denote $\widehat{K}$ as the similarity matrix under these points.Then

$$\begin{aligned}
\widehat{K}_{ij} &= (\widehat{K}_1)_{ij}(\widehat{K}_2)_{ij} \\
&= \text{trace}((x_i^T x_j)(y_j^T y_i)) \\
&= \text{trace}((y_i x_i^T)(x_j y_j^T)) \\
&= \langle x_i y_i^T, x_j y_j^T \rangle_F \\
&= \langle z_i, z_j \rangle_{\mathbb{R}^{n^2}},
\end{aligned}$$

where x_i and y_i are the ith columns of X and Y, respectively, and $z_i = \text{vec}(x_i y_i^T)$. Consequently, K is p.d.. □

Theorem

If K is a p.d. kernel, then so is e^K.

Proof.

$$e^{K(x,x')} = \lim_{n\to\infty} \sum_{i=0}^{n} \frac{K(x,x')^i}{i!}$$

Exercise

Show that $\langle x, x' \rangle_{\mathbb{R}^p}^d$ is p.d. on $\chi = \mathbb{R}^p$ for any $d \in \mathbb{N}$.

Quizz: which of the following are p.d. kernels

- $\mathcal{X} = (-1, 1)$, $K(x, x') = \dfrac{1}{1 - xx'}$
- $\mathcal{X} = \mathbb{N}$, $K(x, x') = 2^{x+x'}$
- $\mathcal{X} = \mathbb{N}$, $K(x, x') = 2^{xx'}$
- $\mathcal{X} = \mathbb{R}_+$, $K(x, x') = \log(1 + xx')$

p.d. Kernels as inner products

Theorem (Aronszajn[a], 1950)

[a] Nachman Aronszajn (1907- 1980), Polish American mathematician, systematically developed the concept of reproducing kernel Hilbert space (Hilbert space)

K is a p.d. kernel on the set χ if and only if there exists a Hilbert space $\mathcal{H}$ and a mapping

$$\Phi : \chi \to \mathcal{H}$$

such that for any $x, x' \in \chi$, there holds

$$K(x, x') = \langle \Phi(x), \Phi(x') \rangle_{\mathcal{H}}.$$

Proof: finite case

- Assume $\mathcal{X} = \{x_1, \cdots, x_N\}$ is finite of size N
- Any p.d. kernel $K : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ is entirely defined by the $N \times N$ symmetric p.s.d. matrix $K_{ij} := K(x_i, x_j)$.
- It can therefore be diagonalized on an orthonormal basis of eigenvectors $(u_1, \cdots, u_N)$ with nonnegative eigenvalues $0 \leq \lambda_1 \leq \cdots \leq \lambda_N$, i.e.,

$$K(x_i, x_j) = \left(\sum_{\ell=1}^{N} \lambda_\ell u_\ell u_\ell^T \right)_{ij} = \sum_{\ell=1}^{N} \lambda_\ell (u_\ell)_i (u_\ell)_j = \langle \phi(x_i), \phi(x_j) \rangle_{\mathbb{R}^N}$$

with

$$\phi(x_i) = \begin{bmatrix} \sqrt{\lambda_1}(u_1)_i \\ \vdots \\ \sqrt{\lambda_N}(u_N)_i \end{bmatrix}.$$

Proof: general case

- Mercer (1909) for $\mathcal{X} = [a, b] \subset \mathbb{R}$ (more generally $\mathcal{X}$ compact) and K continuous.
- Kolmogorov (1941) for $\mathcal{X}$ countable
- Aronszajn (1944, 1950) for the general case.

The proof of the general case requires introducing the concept of Reproducing Kernel Hilbert Spaces (Hilbert space), which we will skip in this course.

Inner product in $\mathcal{H}$

The feature map $\Phi = K(x, \cdot)$, and thus

$$\langle K(x, \cdot), K(x', \cdot) \rangle_{\mathcal{H}} = K(x, x').$$

This means the inner produce in $\mathcal{H}$ is determined by the kernel K directly.

kernel 先规定了“相似度应该是多少”，然后 RKHS 的内积被构造出来去匹配这个相似度。

Summary: p.d. Kernels

- We introduced the defn. of p.d. kernels, with many examples,
  - the simplest p.d. kernel on $\mathbb{R} \times \mathbb{R}$
  - the simplest p.d. kernel on $\mathbb{R}^d \times \mathbb{R}^d$
  - polynomial kernel
  - combining kernel
- Aronszajn's theorem
  - relates a p.d. kernel with a Hilbert space and a feature map

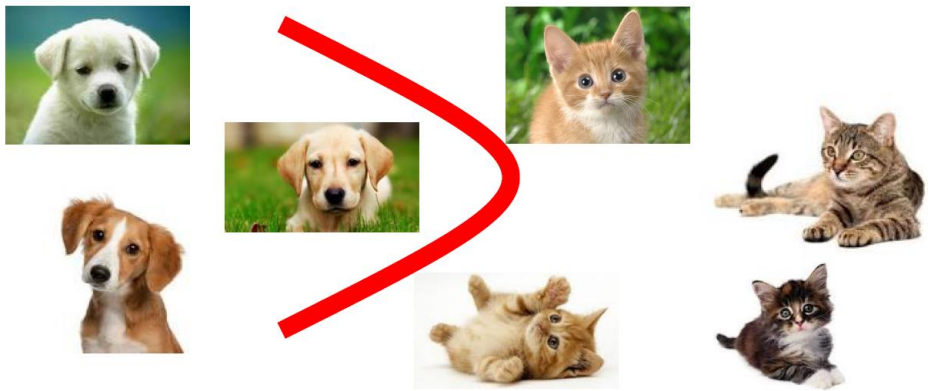
Example: classification

Task: recognize if an image is a dog or a cat

- $\mathcal{X}$ = set of images ($\mathbb{R}^d$)
- $\mathcal{Y} = \{\text{cat}, \text{dog}\}$

Goal is to learn a prediction function $f : \mathcal{X} \to \mathcal{Y}$ such that

$$\min_{f \in \mathcal{F}} \underbrace{\frac{1}{n} \sum_{i=1}^{n} \ell(f(x_i), y_i)}_{\text{empirical risk, data fit}} + \underbrace{\lambda \Omega(f)}_{\text{regularization}}$$



Example from supervised learning

Given

- $\mathcal{X}$, a space of inputs
- $\mathcal{Y}$, a space of outputs
- $(x_i, y_i)_{i=1,\cdots,n}$, a training set of (input, output) pairs,

Goal is to learn a prediction function $f : \mathcal{X} \to \mathcal{Y}$ such that

$$\min_{f \in \mathcal{F}} \underbrace{\frac{1}{n} \sum_{i=1}^{n} \ell(f(x_i), y_i)}_{\text{empirical risk, data fit}} + \underbrace{\lambda \Omega(f)}_{\text{regularization}}$$

Depending on the nature of the output, this covers,

- Regression, when $\mathcal{Y} = \mathbb{R}$
- Classification, when $\mathcal{Y} = \{-1, 1\}$ or any set of two labels
- Structured output regression or classification when $\mathcal{Y}$ is more general

Example from supervised learning

Given

- $\mathcal{X}$, a space of inputs
- $\mathcal{Y}$, a space of outputs
- $(x_i, y_i)_{i=1,\cdots,n}$, a training set of (input, output) pairs,

Goal is to learn a prediction function $f : \mathcal{X} \to \mathcal{Y}$ such that

$$\min_{f \in \mathcal{F}} \underbrace{\frac{1}{n} \sum_{i=1}^{n} \ell(f(x_i), y_i)}_{\text{empirical risk, data fit}} + \underbrace{\lambda \Omega(f)}_{\text{regularization}}$$

- $\mathcal{F}$ = arbitrary function class
- $\Omega(f)$ = arbitrary complexity penalty

This is **too abstract** to compute with.

Example with linear models: logistic regression, etc.

- assume a linear relation between y and features x
- $f(x) = w^T x + b$ is parameterized by w, b
- the loss function $\ell(\cdot, \cdot)$ is often convex
- The norm $\Omega(\cdot)$ is often the squared ℓ_2-norm

Motivation from supervised learning

In the kernel methods, we learn the target function f from a Hilbert space $\mathcal{H}$ with the kernel map ϕ, i.e., we solve

$$\min_{f \in \mathcal{H}} \underbrace{\frac{1}{n} \sum_{i=1}^{n} \ell(f(x_i), y_i)}_{\text{empirical risk, data fit}} + \underbrace{\lambda \|f\|_{\mathcal{H}}}_{\text{regularization}}$$

with

$$f(x) := \langle \phi(x), f \rangle_{\mathcal{H}}.$$

Kernel methods turn supervised learning into linear learning in a Hilbert space, with regularization becoming a norm penalty. Learning a function becomes learning a vector f in a Hilbert space.

Linear forms

Kernel methods map data in $\mathcal{X}$ to Hilbert space $\mathcal{H}$ and admit a linear form.

Motivation from supervised learning

In the kernel methods, we learn the target function f from a Hilbert space $\mathcal{H}$ with the kernel map ϕ, i.e., we solve

$$\min_{f \in \mathcal{H}} \underbrace{\frac{1}{n} \sum_{i=1}^{n} \ell(f(x_i), y_i)}_{\text{empirical risk, data fit}} + \underbrace{\lambda \|f\|_{\mathcal{H}}}_{\text{regularization}} .$$

First purpose: embed data into a vectorial space, where

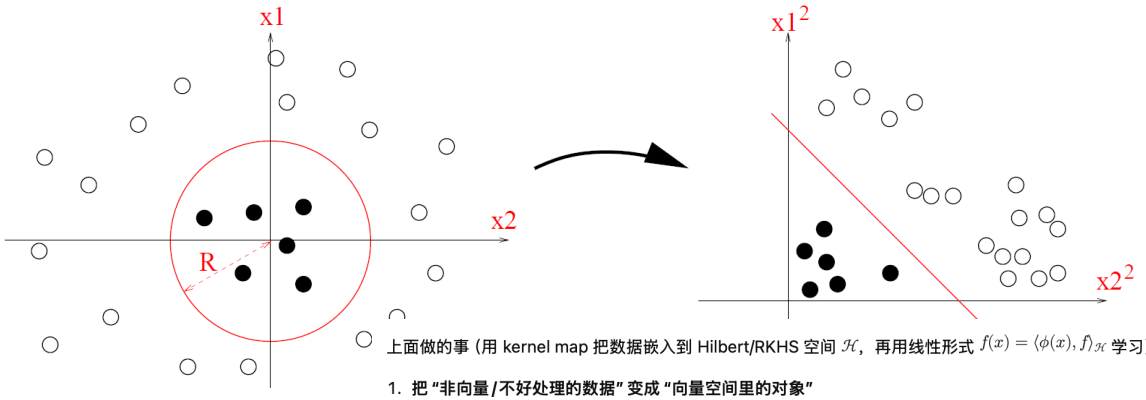
- many geometrical operations exist (angle computation, projection on linear subspaces, definition of barycenters, etc.)
- one may learn potentially rich infinite-dimensional models
- regularization is natural and theoretically grounded

The principle is generic and does not assume anything about the nature of the set $\mathcal{X}$.

Motivation from supervised learning

Second purpose: unhappy with the current Euclidean structure?

- lift data to a higher-dimensional space with nicer properties (e.g., linear separability, clustering structure)
- then, the linear form $f(x) := \langle \phi(x), f \rangle_{\mathcal{H}}$ in $\mathcal{H}$ may correspond to a nonlinear model in $\mathcal{X}$.



上面做的事（用 kernel map 把数据嵌入到 Hilbert/RKHS 空间 $\mathcal{H}$，再用线性形式 $f(x) = \langle \phi(x), f \rangle_{\mathcal{H}}$ 学习）目的可以很精炼地概括成两点：

1. **把“非向量/不好处理的数据”变成“向量空间里的对象”**
   进了 $\mathcal{H}$ 之后，你立刻能用几何工具：角度、投影、子空间、均值/重心等；同时可以用非常丰富（甚至无限维）的特征表示来建模。

2. **在高维空间做线性学习 = 在原空间得到非线性模型**
   如果原来的欧氏结构不适合（不可分、聚类结构不好等），把数据 lift 到更“好用”的空间里，线性模型在 $\mathcal{H}$ 里很简单，但回到原空间 $\mathcal{X}$ 往往对应强大的非线性决策/回归函数。

附带的关键收益：**正则化变得自然且有理论支撑**——直接用 $\|f\|_{\mathcal{H}}$ 控制复杂度，防止高维下过拟合。

The kernel trick

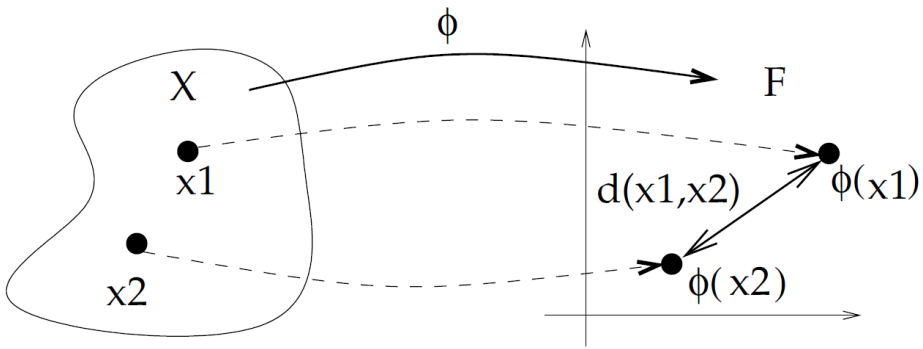
Proposition

Any algorithm to process finite-dimensional vectors that can be expressed only in terms of pairwise inner products can be applied to potentially infinite-dimensional vectors in the feature space of a p.d. kernel by replacing each inner product evaluation by a kernel evaluation.

- There is no need to use the inner product in the Hilbert space
- There is no need to know the kernel map

Example: computing distances in the feature space

- The kernel trick lets us **implicitly work in high-dimensional spaces** by replacing dot products with kernel evaluations.
- It avoids the need to explicitly compute feature maps, making complex non-linear models feasible.
- The distance formula in your slide is a perfect example: we used only kernel functions to compute a distance in feature space, without ever seeing $\phi(x)$.



$$
\begin{aligned}
d_K(x_1, x_2)^2 &= \|\phi(x_1) - \phi(x_2)\|_{\mathcal{H}}^2 \\
&= \langle \phi(x_1) - \phi(x_2), \phi(x_1) - \phi(x_2) \rangle_{\mathcal{H}} \\
&= \langle \phi(x_1), \phi(x_1) \rangle_{\mathcal{H}} + \langle \phi(x_2), \phi(x_2) \rangle_{\mathcal{H}} - 2\langle \phi(x_1), \phi(x_2) \rangle_{\mathcal{H}} \\
&= K(x_1, x_1) + K(x_2, x_2) - 2K(x_1, x_2).
\end{aligned}
$$

Summary: kernel trick

- The kernel trick is a trivial statement with important applications.
- It can be used to obtain nonlinear versions of well-known linear algorithms, e.g., by replacing the classical inner product by a Gaussian kernel.
- It can be used to apply classical algorithms to non vectorial data (e.g., strings, graphs) by again replacing the classical inner product by a valid kernel for the data.
- It allows in some cases to embed the initial space to a larger feature space and involve points in the feature space with no pre-image (e.g., barycenter).

Representer Theorem: Motivation

- An Hilbert space is a space of (potentially nonlinear) functions, and $\|f\|_{\mathcal{H}}$ measures the smoothness of f
- Given a set of data $(x_i, y_i)_{i=1,\cdots,n}$, a training set of (input, output) pairs, a natural way to estimate a regression function $f : \mathcal{X} \to \mathbb{R}$ is to solve

$$\min_{f \in \mathcal{H}} \underbrace{\frac{1}{n} \sum_{i=1}^{n} \ell(f(x_i), y_i)}_{\text{empirical risk, data fit}} + \underbrace{\lambda \|f\|_{\mathcal{H}}}_{\text{regularization}}$$

  for certain loss function ℓ such as $\ell(y, t) = (y - t)^2$.
- How to solve in practice this problem, potentially infinite-dimension problem?

Representer Theorem

- Let χ be a set endowed with a p.d. kernel K with $\mathcal{H}$ being the corresponding Hilbert space, and let $\mathcal{S} = \{x_1, \cdots, x_n\} \subset \chi$ be a finite subset.
- Let $\Psi : \mathbb{R}^{n+1} \to \mathbb{R}$ be a function of $n+1$ variables, strickly increasing w.r.t. the last variable.

Then any solution to the optimization problem

$$\min_{f \in \mathcal{H}} \Psi(f(x_1), \cdots, f(x_n), \|f\|_{\mathcal{H}})$$

admits a representation of the form,

$$\forall x \in \chi,\ f(x) = \sum_{i=1}^{n} \alpha_i K(x_i, x) = \sum_{i=1}^{n} \alpha_i K_{x_i}(x).$$

In other words, the solution lives in a finite-dimensional subspace,

$$f \in \text{Span}\{K_{x_1}(x), \cdots, K_{x_n}(x)\}.$$

Proof

Part 1.

- Let $\xi(f)$ be the functional that is minimized in the statement of the representer theorem, and $\mathcal{H}_S$ the linear span in $\mathcal{H}$ of the vectors K_{x_i},

$$\mathcal{H}_S = \{f \in \mathcal{H} : f(x) = \sum_{i=1}^{n} \alpha_i K(x_i, x), \text{ for all } (\alpha_1, \cdots, \alpha_n) \in \mathbb{R}^n\}.$$

- $\mathcal{H}_S$ is a finite-dimensional subspace, therefore any function $f \in \mathcal{H}$ can be uniquely decomposed as

$$f = f_S + f_\perp$$

with $f_S \in \mathcal{H}_S$ and $f_\perp \perp \mathcal{H}_S$.

Proof

Part 2.

- Since $\mathcal{H}$ is a Hilbert space, it holds

$$\forall i = 1, \cdots, n : f_{\perp}(x_i) = \langle f_{\perp}, K_{x_i} \rangle_{\mathcal{H}} = 0.$$

Consequently,

$$\forall i = 1, \cdots, n : f(x_i) = f_{\mathcal{S}}(x_i).$$

- Pythagoras' theorem implies

$$\|f\|_{\mathcal{H}}^2 = \|f_{\mathcal{S}}\|_{\mathcal{H}}^2 + \|f_{\perp}\|_{\mathcal{H}}^2.$$

- As a consequence, $\xi(f) \geq \xi(f_{\mathcal{S}})$, with equality if and only if $f_{\perp} = 0$.

Remarks

In the deep learning, the function Ψ usually takes the form,

$$\Psi(f(x_1), \cdots, f(x_n), \|f\|_{\mathcal{H}}) = c(f(x_1), \cdots, f(x_n)) + \Omega(\|f\|_{\mathcal{H}})$$

where $c(\cdot)$ measures the fit of f to a given problem (regression, classification, dimension reduction, etc.), and $\Omega(\cdot)$ is strictly increasing. This formulation has two important consequences,

- Theoretically, the minimization will enforce the norm $\|f\|_{\mathcal{H}}$ to be small, which can be beneficial by ensuring a sufficient level of smoothness for the solution (regularization effect).
- Practically, we know by the representer theorem that the solution lives in a subspace of dimension n, which can lead to efficient algorithms although the Hilbert space itself can be of infinite dimension.

Remarks

Dual interpretations of kernel methods

Most kernel methods have two complementary interpretations,

- A geometric interpretation in the feature space, thanks to the kernel trick. Even when the feature space is large, most kernel methods work in the linear span of the embedding of the points available.
- A functional interpretation, often as an optimization problem over (subsets of) the Hilbert space associated to the kernel.

The representer theorem has important consequences, but it is in fact rather trivial. We are looking for a function $f \in \mathcal{H}$ such that

$$f(x) = \langle K_x, f \rangle_{\mathcal{H}}.$$

The part $f_\perp \perp K_{x_i}$ is thus useless to explain the training data.

1.3 Kernel methods in supervised learning

Supervised learning

Definition (Supervised learning)

Given

- $\mathcal{X}$, a space of inputs
- $\mathcal{Y}$, a space of outputs
- $(x_i, y_i)_{i=1,\cdots,n}$, a training set of (input, output) pairs,

the supervised learning problem is to estimate a function $h : \mathcal{X} \to \mathcal{Y}$ to predict the output for any future input.

Depending on the nature of the output, this covers,

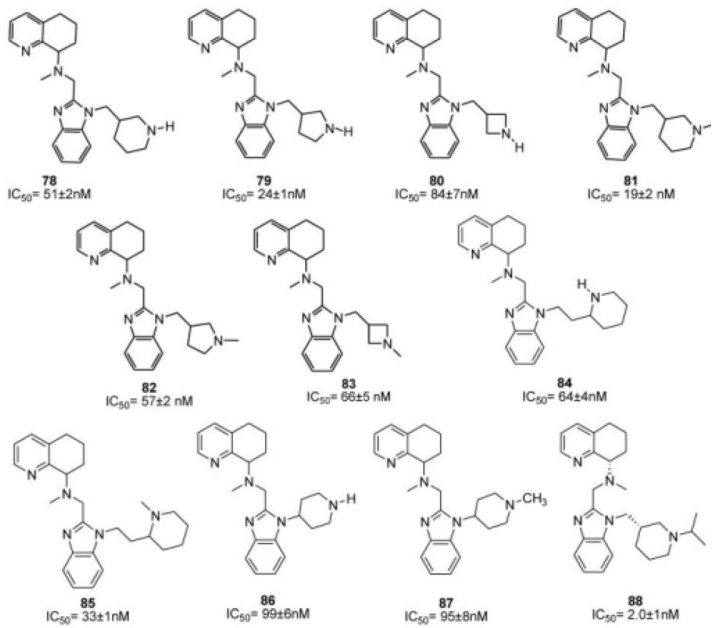
- Regression, when $\mathcal{Y} = \mathbb{R}$
- Classification, when $\mathcal{Y} = \{-1, 1\}$ or any set of two labels
- Structured output regression or classification when $\mathcal{Y}$ is more general

Example: regression

Example

Task: predict the capacity of a small molecule to inhibit a drug target

- $\mathcal{X}$ = set of molecular structures
- $\mathcal{Y} = \mathbb{R}$

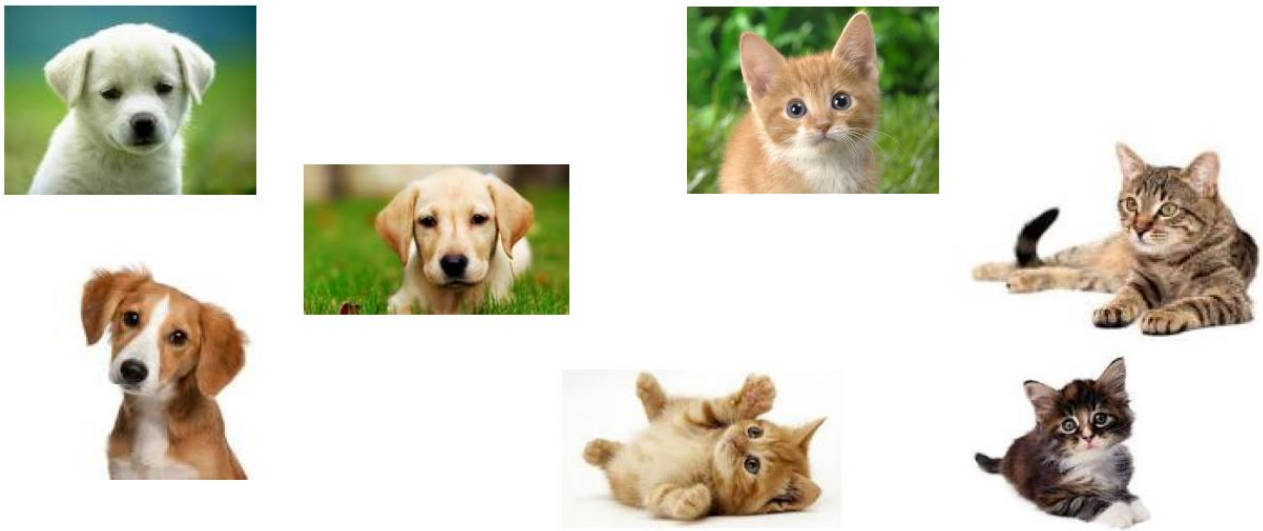


Example: classification

Example

Task: recognize if an image is a dog or a cat

- $\mathcal{X} =$ set of images$(\mathbb{R}^d)$
- $\mathcal{Y} = \{\text{cat}, \text{dog}\}$

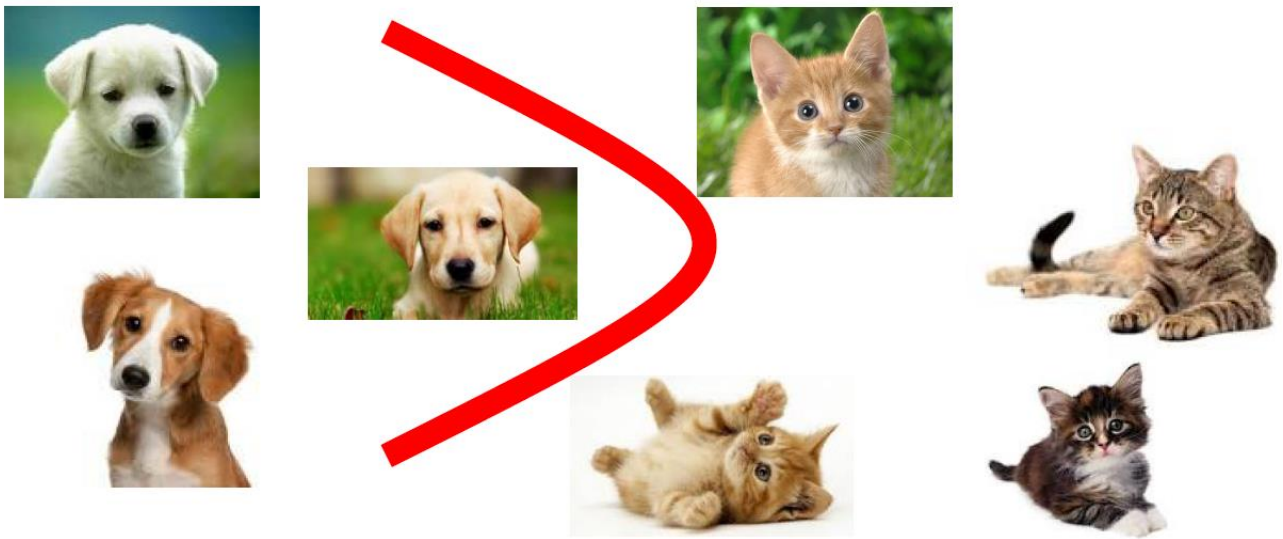


Example: classification

Example

Task: recognize if an image is a dog or a cat

- $\mathcal{X} =$ set of images$(\mathbb{R}^d)$
- $\mathcal{Y} = \{\text{cat}, \text{dog}\}$



Example: structured output

Example

Task: translate from English to Chinese

- $\mathcal{X}$ = finite-length strings of English characters
- $\mathcal{Y}$ = finite-length strings of Chinese characters

Supervised learning with kernels: general principles

Algorithm 1 Supervised learning with kernels

1: Express $h : \mathcal{X} \to \mathcal{Y}$ using a real-valued function $f : \mathcal{Z} \to \mathcal{Y}$

- Regression: $\mathcal{Y} = \mathbb{R}$ and $\mathcal{Z} = \mathcal{Y}$
- Classification: $\mathcal{Y} = \{-1, 1\}$, $\mathcal{Z} = \mathcal{Y}$ and $h(x) = \text{sign}(f(x))$
- Structured output regression or classification: $\mathcal{Y}$ is more general, and

$$h(x) = \arg\max_{y \in \mathcal{Y}} f(x, y) \text{ and } \mathcal{Z} = \mathcal{X} \times \mathcal{Y}$$

2: Define an empirical risk function $R_n(f)$,

$$R_n(f) = \frac{1}{n} \sum_{i=1}^{n} \ell(f(x_i), y_i)$$

3: Define a p.d. kernel on $\mathcal{Z}$ and solve

$$\min_{f \in \mathcal{H}, \|f\|_{\mathcal{H}} \leq B} R_n(f) \text{ or } \min_{f \in \mathcal{H}} R_n(f) + \lambda \|f\|_{\mathcal{H}}$$

Remarks

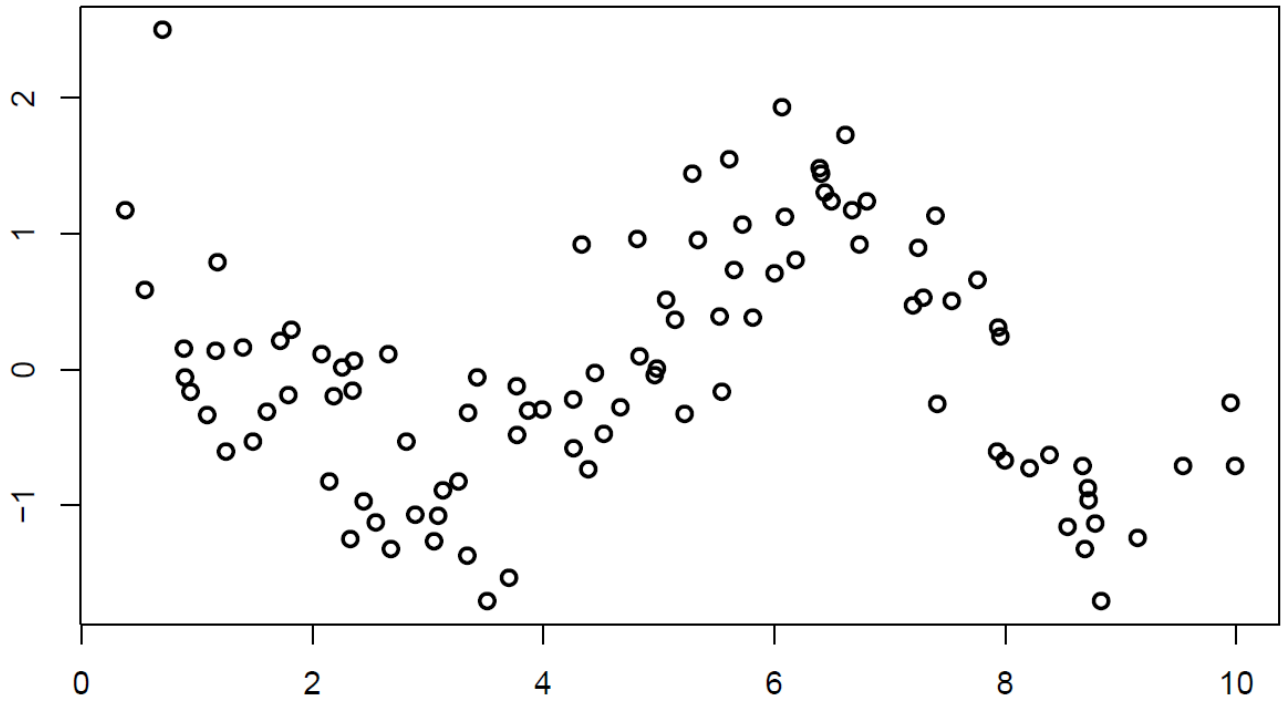
$$\min_{f \in \mathcal{H}} \underbrace{\frac{1}{n} \sum_{i=1}^{n} \ell(f(x_i), y_i)}_{\text{empirical risk, data fit}} + \underbrace{\lambda \|f\|_{\mathcal{H}}}_{\text{regularization}}$$

- Regularization is important to prevent overfitting, especially for high dimensional problems.
- When $\mathcal{Z} = \mathbb{R}^d$ and K is the linear kernel, then f is a linear model.
- Using more general spaces $\mathcal{Z}$ and kernel K allows to
  - learn nonlinear functions over a functional space endowed with a natural regularization
  - learn functions over non-vectorial data such as strings and graphs

Application: Regression

Setup

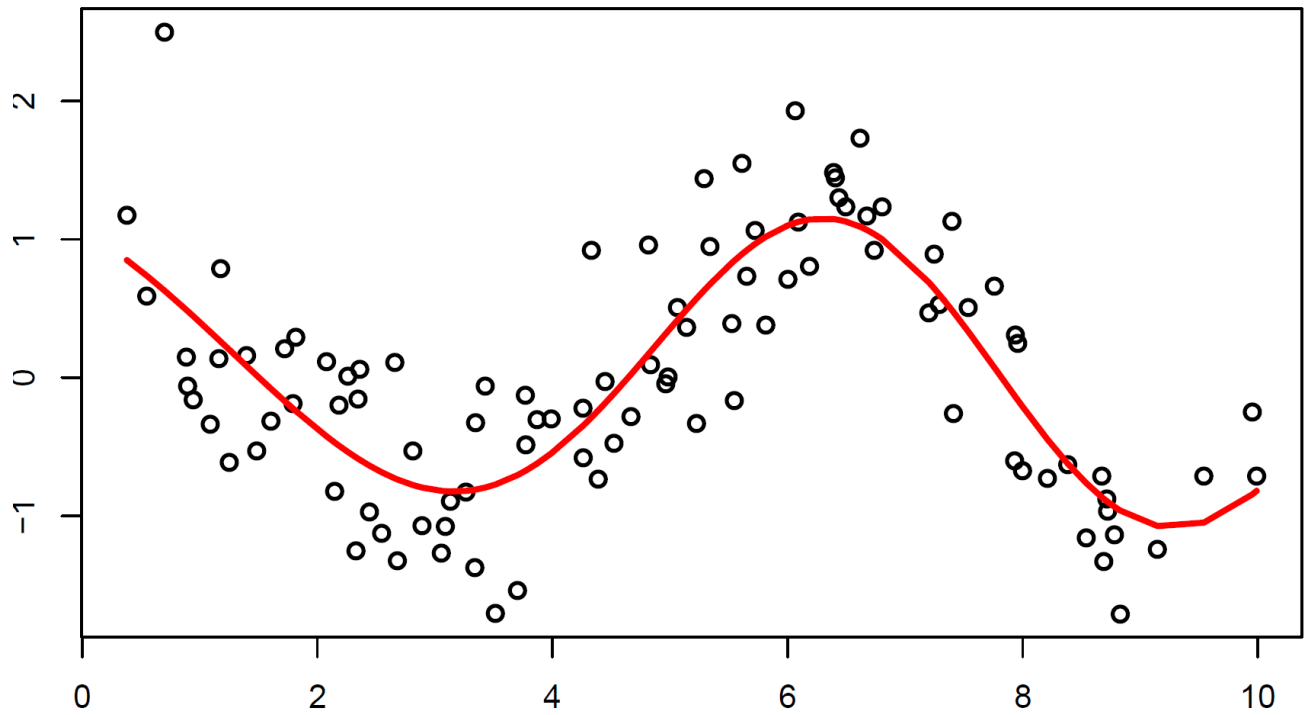
- $\mathcal{X}$: set of inputs
- $\mathcal{Y} := \mathbb{R}$ is real-valued output
- $\mathcal{S}_n = (x_i, y_i)_{i=1,\cdots,n}$ is a training set of n pairs
- Goal is to find $f : \mathcal{X} \to \mathbb{R}$ to predict $f(x)$.



Application: Regression

Setup

- $\mathcal{X}$: set of inputs
- $\mathcal{Y} := \mathbb{R}$ is real-valued output
- $\mathcal{S}_n = (x_i, y_i)_{i=1,\cdots,n}$ is a training set of n pairs
- Goal is to find $f : \mathcal{X} \to \mathbb{R}$ to predict $f(x)$.



Least-square regression over a general functional space

- Quantify the error,

$$\ell(f(x), y) = (y - f(x))^2$$

- Fix a set of functional space $\mathcal{H}$
- Least-square regression amounts to finding $f \in \mathcal{H}$ with the smallest empirical risk, which is mean squared error (MSE) in this case,

$$\hat{f} \in \arg\min_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^{n} \ell(f(x_i), y_i)$$

- Issue: unstable especially in high-dimension, and overfitting when $\mathcal{H}$ is too large.

Kernel ridge regression (KRR)

- Let $\mathcal{H}$ be a Hilbert space, which is associated with a p.d. kernel K on $\mathcal{X}$
- KRR is obtained by regularizing the MSE criterion by the Hilbert space norm,

$$\hat{f} \in \arg\min_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^{n} \ell(f(x_i), y_i) + \lambda \|f\|_{\mathcal{H}}^2 \tag{1}$$

Main feature and advantages

- this prevents overfitting by penalizing nonsmooth functions
- by the representer theorem, any solution takes the following form,

$$\hat{f}(x) = \sum_{i=1}^{n} \alpha_i K(x_i, x),$$

and consequently, this simplifies the solution.

equivalent version: KRR

- Let $\boldsymbol{y} = (y_1, \cdots, y_n)^T \in \mathbb{R}^n$
- Let $\boldsymbol{\alpha} = (\alpha_1, \cdots, \alpha_n)^T \in \mathbb{R}^n$
- Let $\mathbb{K}$ be Gram matrix of size $n \times n$: $\mathbb{K}_{ij} = K(x_i, x_j)$

Then under this setting, we derive,

$$(\hat{f}(x_1), \cdots, \hat{f}(x_n))^T = \mathbb{K}\boldsymbol{\alpha}, \text{ and } \|\hat{f}\|_{\mathcal{H}}^2 = \boldsymbol{\alpha}^T \mathbb{K} \boldsymbol{\alpha}$$

equivalent version: KRR

The KRR problem (1) is equivalent to

$$\arg\min_{\boldsymbol{\alpha} \in \mathbb{R}^n} \frac{1}{n}(\mathbb{K}\boldsymbol{\alpha} - \boldsymbol{y})^T(\mathbb{K}\boldsymbol{\alpha} - \boldsymbol{y}) + \lambda \boldsymbol{\alpha}^T \mathbb{K} \boldsymbol{\alpha}. \tag{2}$$

Solving equivalent version (2)

Equivalent version: KRR

$$\arg\min_{\boldsymbol{\alpha}\in\mathbb{R}^n} \frac{1}{n}(\mathbb{K}\boldsymbol{\alpha}-\boldsymbol{y})^T(\mathbb{K}\boldsymbol{\alpha}-\boldsymbol{y})+\lambda\boldsymbol{\alpha}^T\mathbb{K}\boldsymbol{\alpha}.$$

The objective function is convex and differentiable w.r.t. the parameter α. Consequently, its minimum can be obtained by making the gradient vanishing.

Exercise

Let $F:\mathbb{R}^n\to\mathbb{R}^+$ given by $F(\boldsymbol{\alpha}):=\frac{1}{n}(\mathbb{K}\boldsymbol{\alpha}-\boldsymbol{y})^T(\mathbb{K}\boldsymbol{\alpha}-\boldsymbol{y})+\lambda\boldsymbol{\alpha}^T\mathbb{K}\boldsymbol{\alpha}$.
Calculate ∇F.

$$\nabla F = \frac{2}{n}\mathbb{K}\left((\mathbb{K} + \lambda n\mathbb{I})\boldsymbol{\alpha} - \boldsymbol{y}\right).$$

Since $\lambda > 0$ and $\mathbb{K}$ is positive semidefinite, $\mathbb{K} + \lambda n\mathbb{I}$ is invertible, therefore, we derive

$$\nabla F = \mathbf{0} \longrightarrow \boldsymbol{\alpha} = \left(\mathbb{K} + \lambda n\mathbb{I}\right)^{-1}\boldsymbol{y} + \mathrm{Ker}(\mathbb{K}).$$

Quiz

Is the minimizer for KRR problem unique?

Recall that the minimizer $\hat{f}(x) = \sum_{i=1}^{n} K(x, x_i)\alpha_i$ with

$$\mathbb{K}\left((\mathbb{K} + \lambda n \mathbb{I})\boldsymbol{\alpha} - \boldsymbol{y}\right) = \mathbf{0}.$$

However, this linear system can have multiple solutions, since

$$(\mathbb{K} + \lambda n \mathbb{I})\boldsymbol{\alpha} - \boldsymbol{y} \in \mathrm{Ker}(\mathbb{K}). \tag{3}$$

Hint.

- Since the gram matrix $\mathbb{K}$ is symmetric, it can be diagonalized in an orthonormal basis, and $\mathrm{Ker}(\mathbb{K}) \perp \mathrm{Im}(\mathbb{K})$.
- $\mathrm{Ker}(\mathbb{K})$ and $\mathrm{Im}(\mathbb{K})$ are invariant under the matrix $(\mathbb{K} + \lambda n \mathbb{I})^{-1}$, therefore, (3) is equivalent to

$$\boldsymbol{\alpha} - (\mathbb{K} + \lambda n \mathbb{I})^{-1}\boldsymbol{y} \in \mathrm{Ker}(\mathbb{K}).$$

Let $\mathbb{K}$ be positive semi-definite and $n > 0$, prove that

$$\mathbb{K}(\mathbb{K} + \lambda n\mathbb{I})^{-1} = (\mathbb{K} + \lambda n\mathbb{I})^{-1}\mathbb{K}.$$

1/2.

Since $\mathbb{K}$ is positive semi-definite, then we can represent $\mathbb{K}$ as

$$\mathbb{K} = \mathbb{V}^{-1}\mathbf{\Sigma}\mathbb{V}$$

with $\mathbb{V} \in \mathbb{R}^{n\times n}$ and $\mathbf{\Sigma}$ being a diagomal matrix with nonnegative diagonals. Hence[a],

$$\mathbb{K} + \lambda n\mathbb{I} = \mathbb{V}^{-1}(\mathbf{\Sigma} + n\mathbb{I})\mathbb{V}, \text{ and therefore,}$$
$$(\mathbb{K} + \lambda n\mathbb{I})^{-1} = \mathbb{V}^{-1}(\mathbf{\Sigma} + n\mathbb{I})^{-1}\mathbb{V}.$$

□

[a] If you want to show $\mathbb{K}^m = \mathbb{V}^{-1}\mathbf{\Sigma}^m\mathbb{V}$ for $m \in \mathbb{N}^+$, please refer to the Spectral Mapping Theorem

2/2.

Consequently, we obtain

$$\begin{aligned}\mathbb{K}(\mathbb{K}+\lambda n\mathbb{I})^{-1} &= \mathbb{V}^{-1}\Sigma(\Sigma+n\mathbb{I})^{-1}\mathbb{V} = \mathbb{V}^{-1}(\Sigma+n\mathbb{I})^{-1}\Sigma\mathbb{V} \\ &= (\mathbb{K}+\lambda n\mathbb{I})^{-1}\mathbb{K}.\end{aligned}$$

□

Exercise

Let $\boldsymbol{\alpha}^{(1)} = (\mathbb{K} + \lambda n \mathbb{I})^{-1} \boldsymbol{y}$, and $\boldsymbol{\alpha}^{(2)} = \boldsymbol{\alpha}^{(1)} + \boldsymbol{v}$ for any $\boldsymbol{v} \in \text{Ker}(\mathbb{K})$. Then $\nabla F(\boldsymbol{\alpha}^{(j)}) = \mathbf{0}$ for $j = 1, 2$. Let

$$f^{(j)} = \sum_{i=1}^{n} K(x, x_i) \boldsymbol{\alpha}_i^{(j)} \text{ for } j = 1, 2.$$

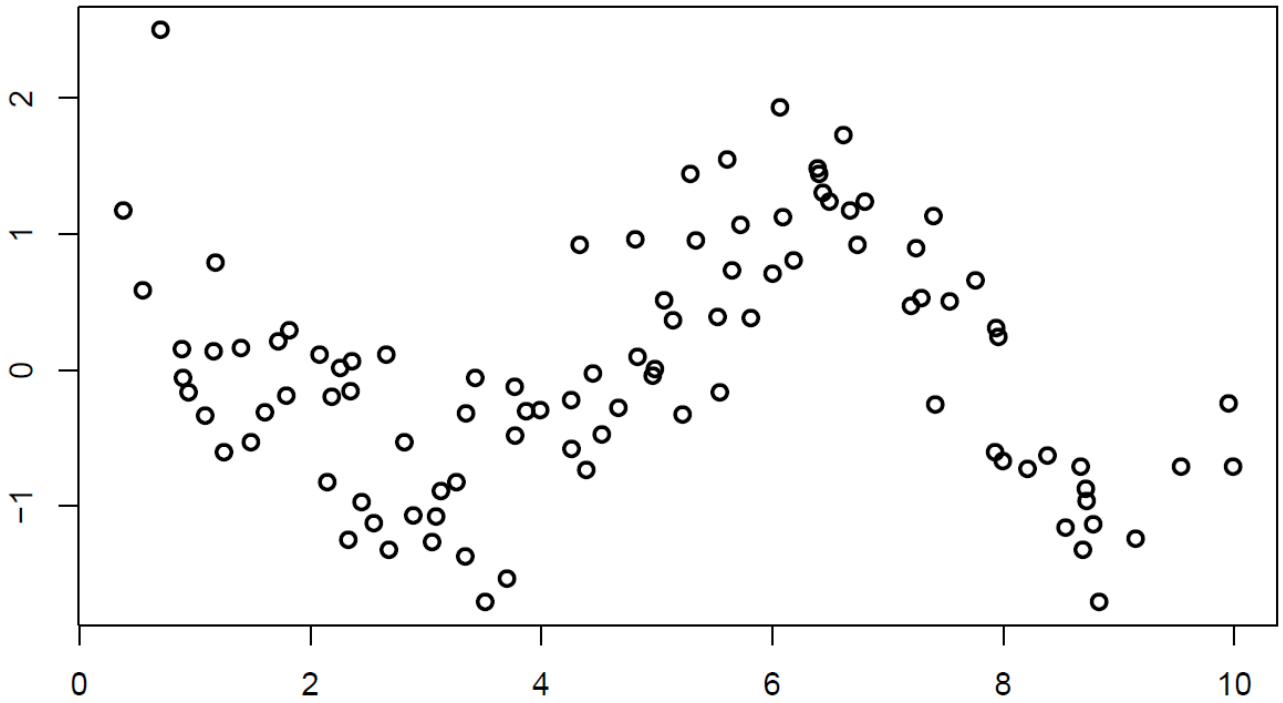
Then $F(\boldsymbol{\alpha}^{(1)}) = \boldsymbol{\alpha}^{(2)}$ and $f^{(1)}(x_i) = f^{(2)}(x_i)$ for $i = 1, \cdots, n$.

Conclusion

Even though the minimizer of KRR is not unique, but they give the same value in the objective function, and they have no difference on the train data.

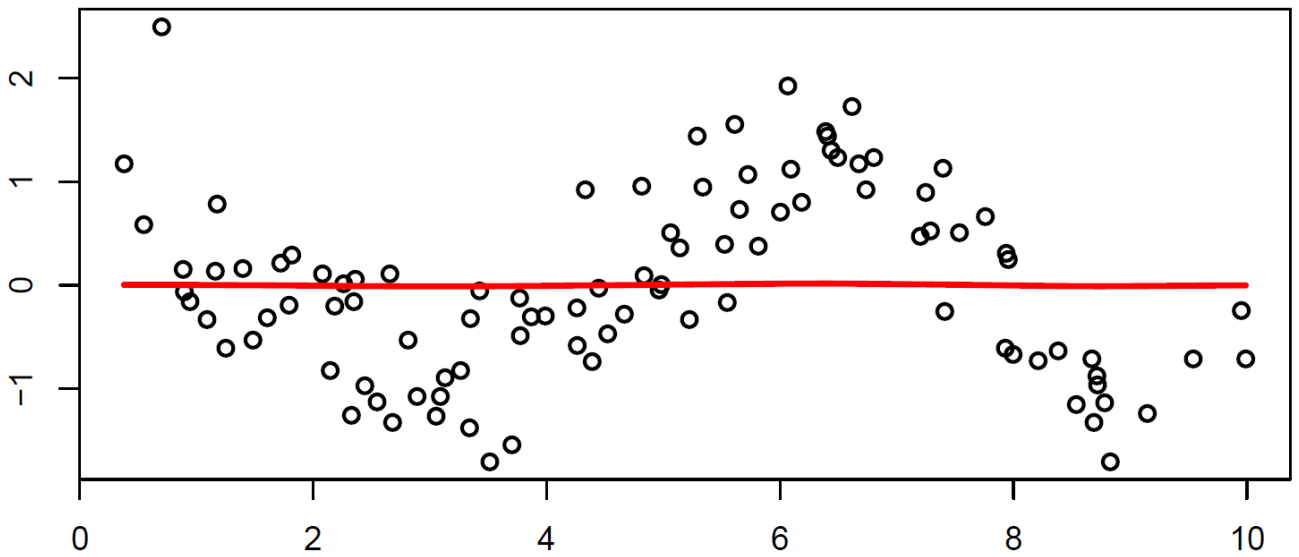
Application: KRR with Gaussian RBF kernel

$$K(\boldsymbol{x}, \boldsymbol{x}') = \exp(-\gamma(\boldsymbol{x}, \boldsymbol{x}')_{\ell_2}) \text{ for all } \boldsymbol{x}, \boldsymbol{x}' \in \mathbb{R}^d.$$



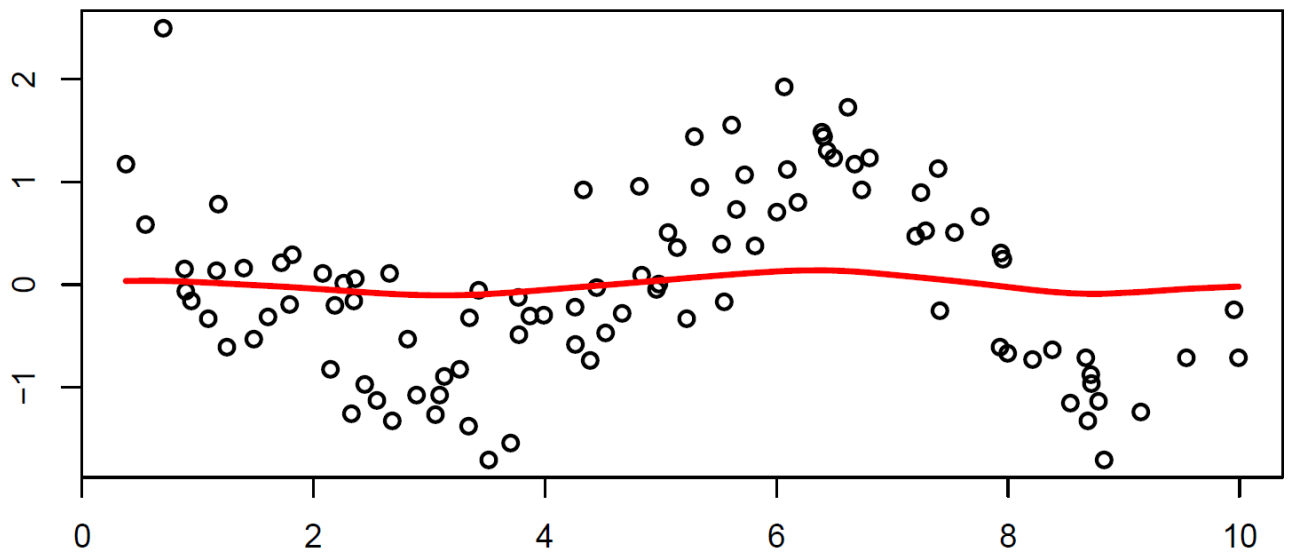
Application: KRR with Gaussian RBF kernel

Figure: $\lambda = 1000$



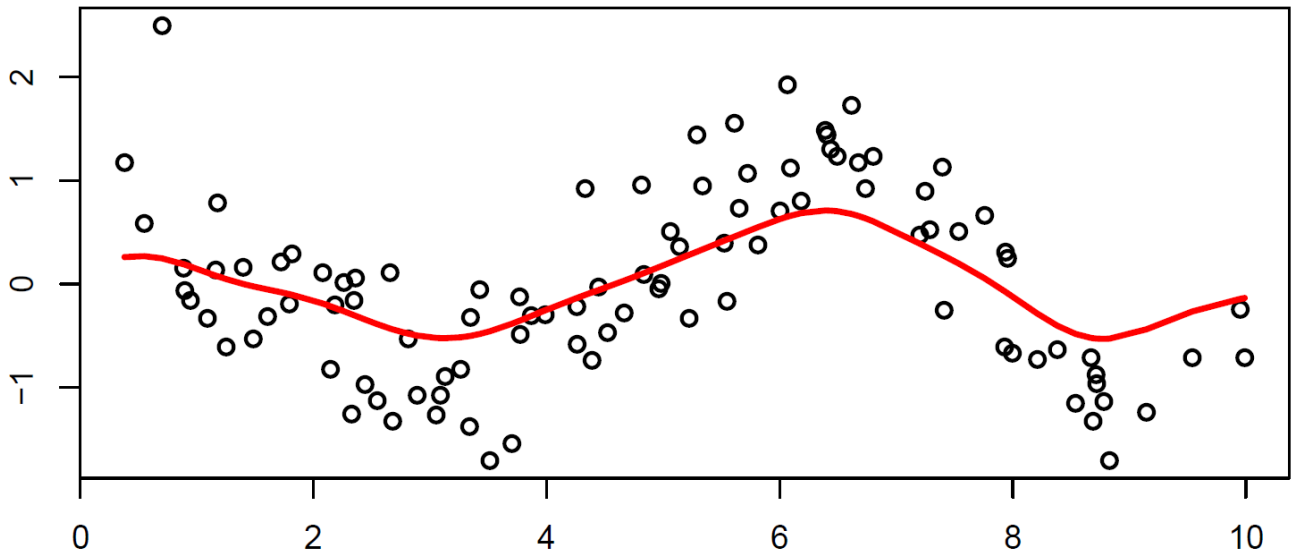
Application: KRR with Gaussian RBF kernel

Figure: $\lambda = 100$



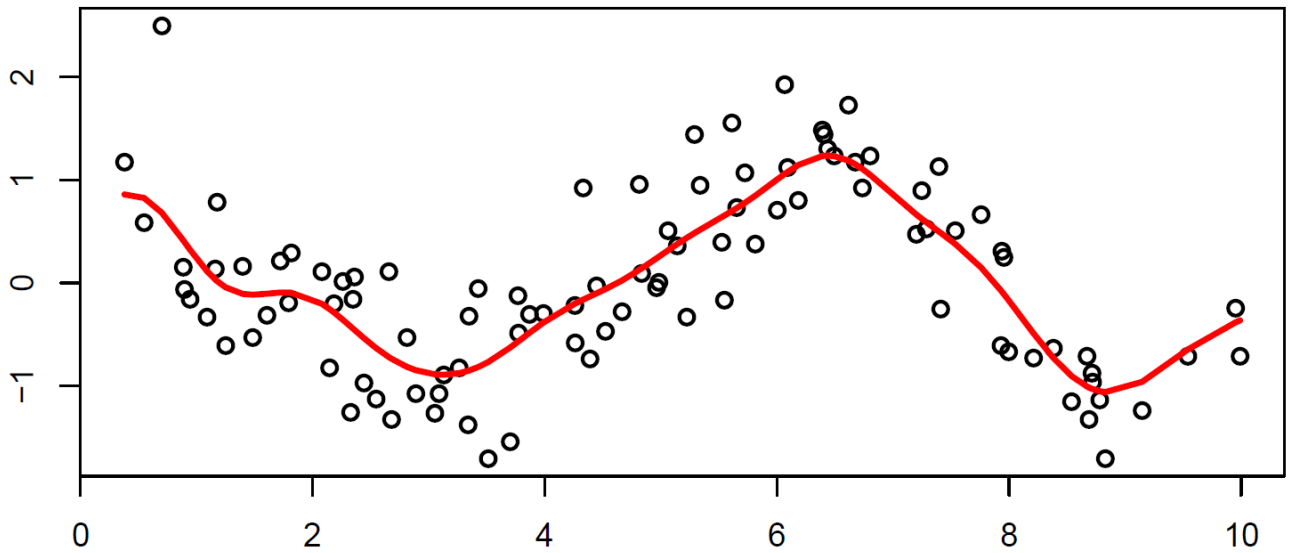
Application: KRR with Gaussian RBF kernel

Figure: $\lambda = 10$



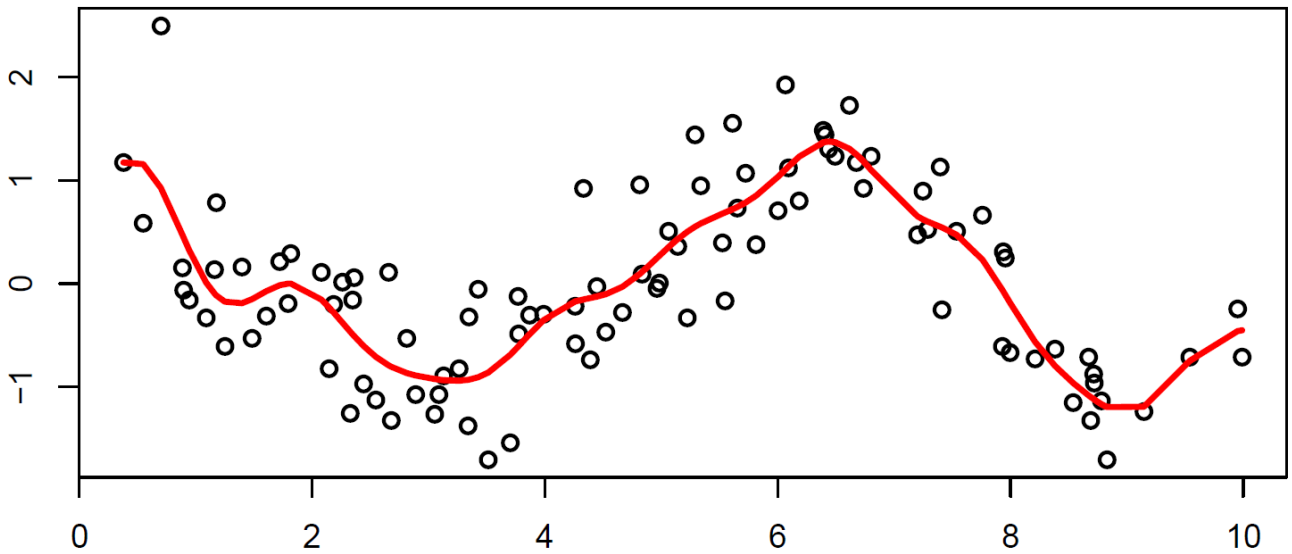
Application: KRR with Gaussian RBF kernel

Figure: $\lambda = 1$



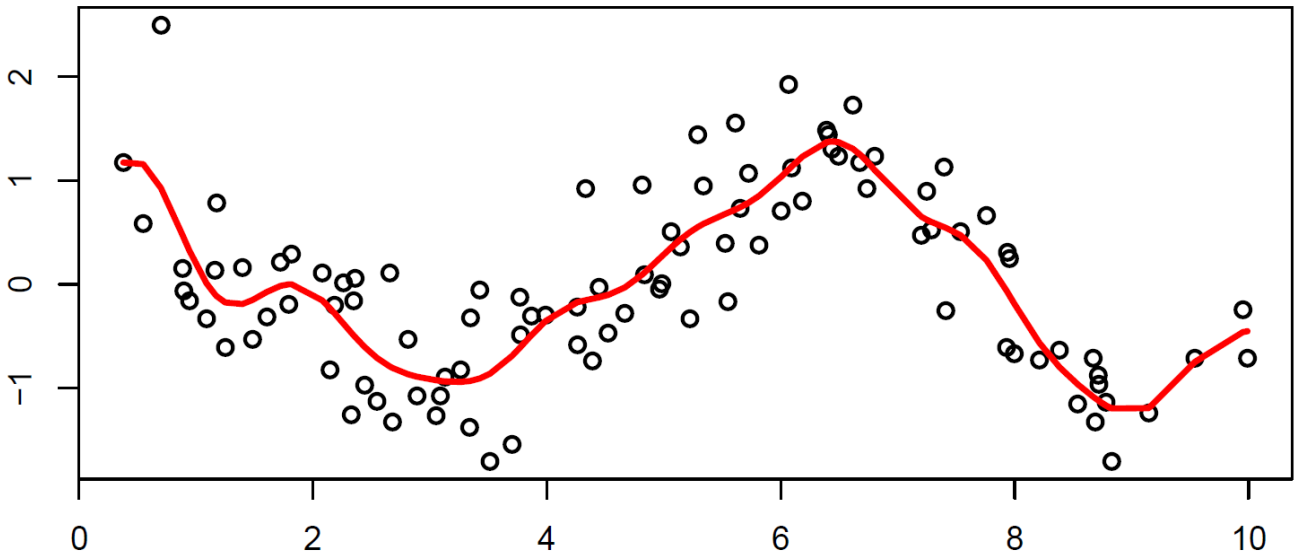
Application: KRR with Gaussian RBF kernel

Figure: $\lambda = 0.1$



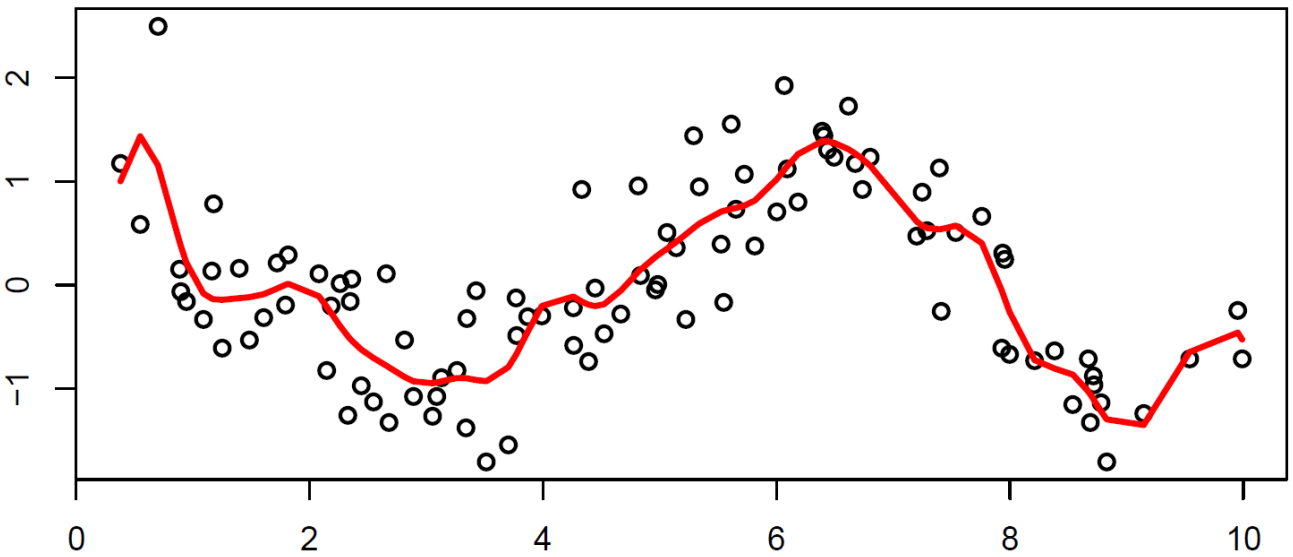
Application: KRR with Gaussian RBF kernel

Figure: $\lambda = 0.01$



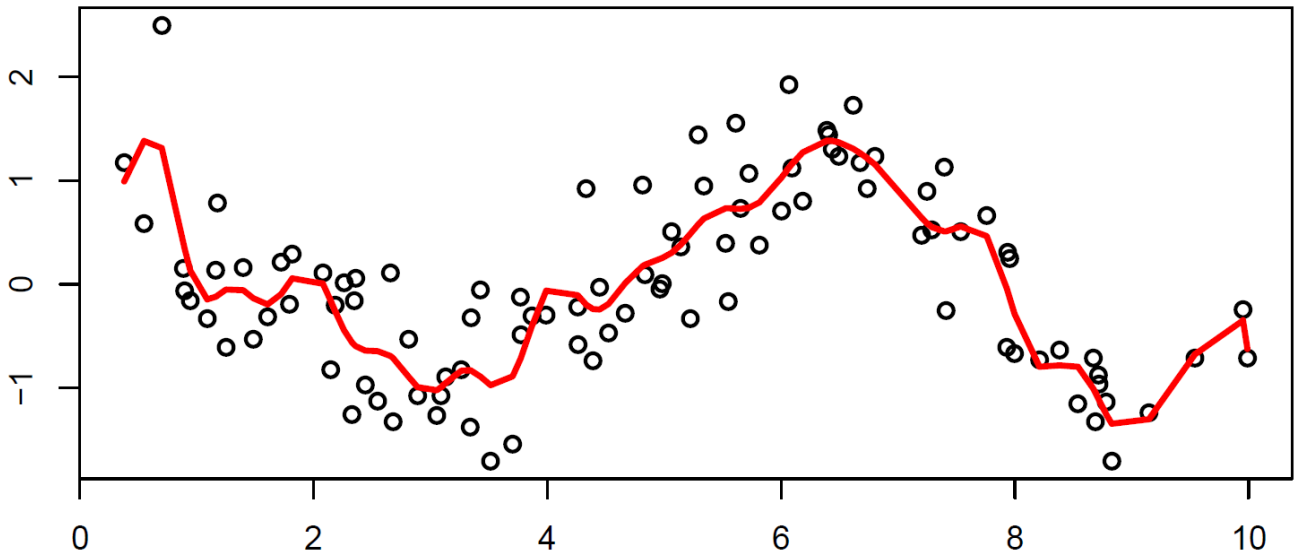
Application: KRR with Gaussian RBF kernel

Figure: $\lambda = 0.001$



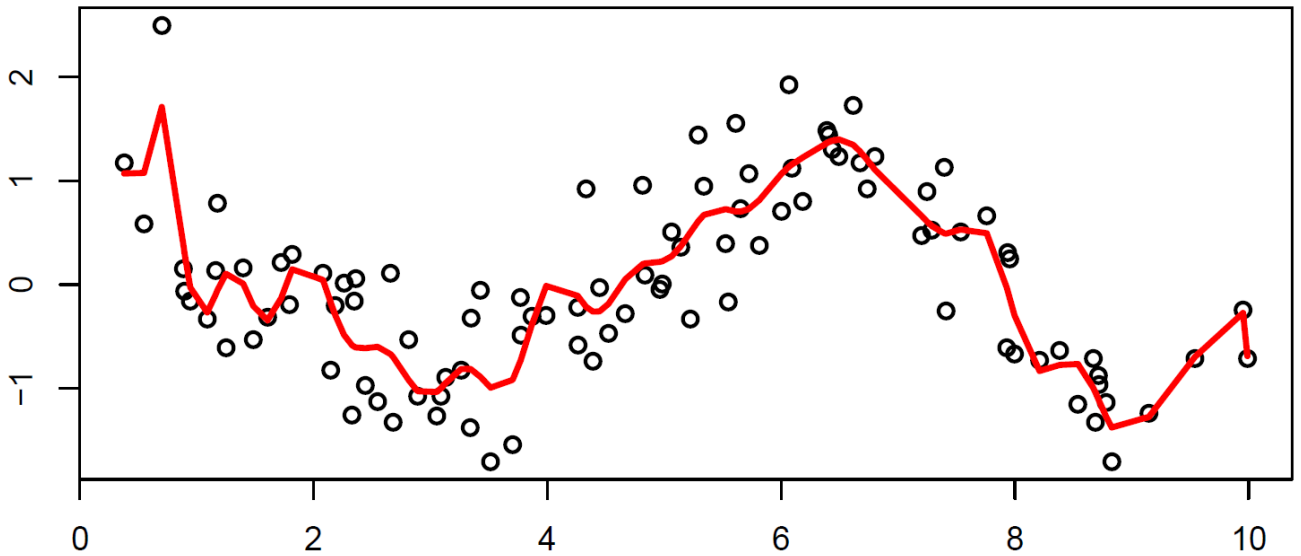
Application: KRR with Gaussian RBF kernel

Figure: $\lambda = 0.0001$



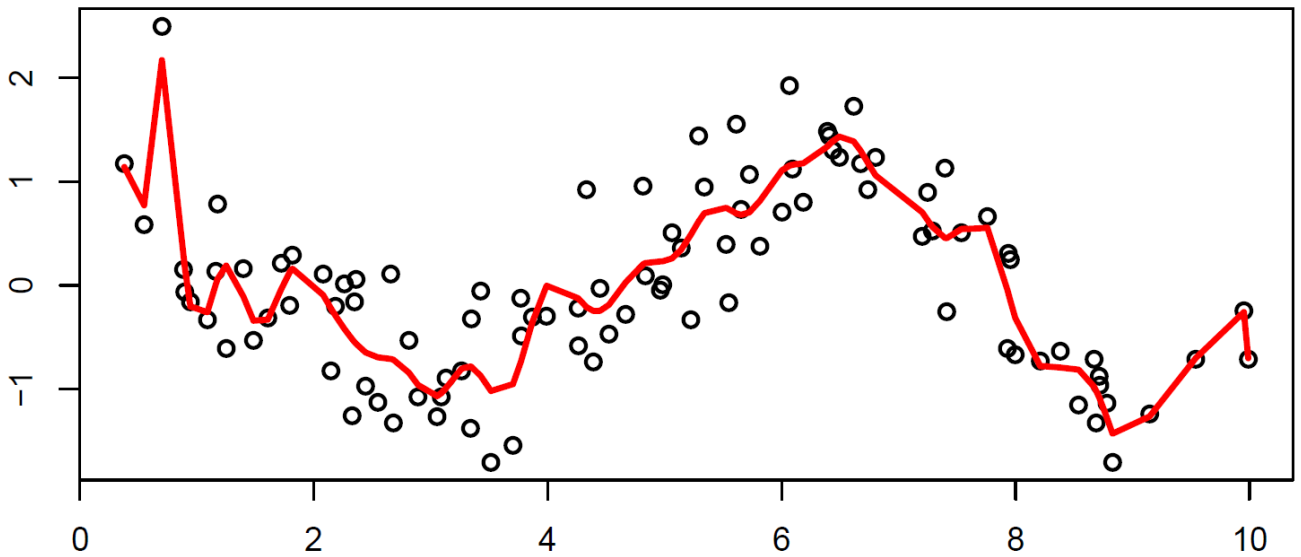
Application: KRR with Gaussian RBF kernel

Figure: $\lambda = 0.00001$



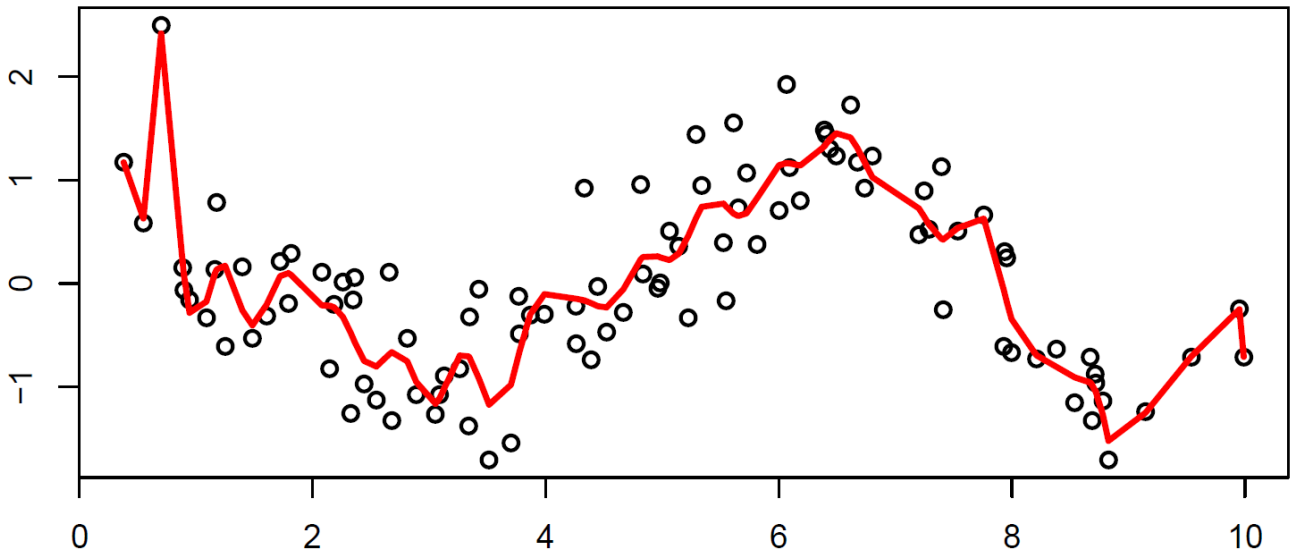
Application: KRR with Gaussian RBF kernel

Figure: $\lambda = 0.000001$



Application: KRR with Gaussian RBF kernel

Figure: $\lambda = 0.0000001$



Summary

- Kernel methods can be applied to almost any supervised learning tasks.
- However, how to choose the kernel has been not well understood yet.
- It is also not well known how to choose the regularization parameter.